Towards a proposition for the unsupervised classification of hyperspectral Images

Gabriel Dauphin

November 9, 2023

1 Introduction

In the literature, when classifying hyperspectral images, most papers take into account not only the spectral content but also the spatial content. To do so, the main methodology is to consider the spectral content of neighboring pixels as complementary information enriching the features. Its description may look like the combination of two different techniques, one being more sensitive to the spectral content of neighboring pixels. An other methodology is to assume that the captured spectral content of a given pixel could be the combination of a small number of the spectral content of some endmembers. A methodology that is both different and not opposed, exploits some contextual information, namely the spatial shape of the classified regions are expected to display some spatial continuity. Technically a simple model assumes that spatially neighboring pixels are very likely to belong to the same region, usually with the help of techniques derived from Markov Random Fields. This viewpoint yields very appealing classification maps, and less appealing metric-measured performances.

My standpoint fits in the following objective: the expected spatial shape of classified regions is a precious source of information, but one that is difficult to exploit. More specifically to collect that source of information, I am presently claiming:

1. Linear spatial filters applied on pixel intensities may not be appropriate tools.
2. Combining the likelihood of some clusters with assumptions on their spatial shapes does not yield appropriate tools.
3. The appropriate tools need to make important exchanges of information between the analysis of the spectral content and that of the spatial content.
4. The predicted classified regions should be relevant both in their collective spectral contents and in their spatial shapes.
5. The main limit for improving performance in classification is not so much the amount of ground truth, than the numerical complexity and possibly the amount of coregistered hyperspectral images. Numerical complexity is an important issue, which if not overcome, may result in the impossibility to exploit that source of information.

This document is not based on an important investigation in the literature, I am referring to what I remember having read. Unfortunately, I am not proving these claims, these should be rather considered as research prospects. My aim in this document is to expose precisely those prospects with some experiments. I expect this document to be helpful in choosing knowingly the research areas to investigate.

Section 2 is focused on accounting for claim 1 using correlated pairs of random variables. Section 3 and 4 define the clustering problem and the proposed evaluation techniques. Section 5 is an attempt to account for claim 2 with a simplistic use of the spatial context. Section 6 describes a clustering technique in line with claim 4, its algorithm illustrates claim 3. The results are non-conclusive: performances are not significantly improved, the proposed evaluation technique seems to be inaccurate and the combination of the clustering technique and the exploring technique have not succeeded in maximizing the spatial continuity. It is with these difficulties in mind, that I am, nonetheless, making a proposal, described in section 7. It is focused on reducing the numerical complexity, its success would account for claim 5. It uses spatial metrics described in section 8.

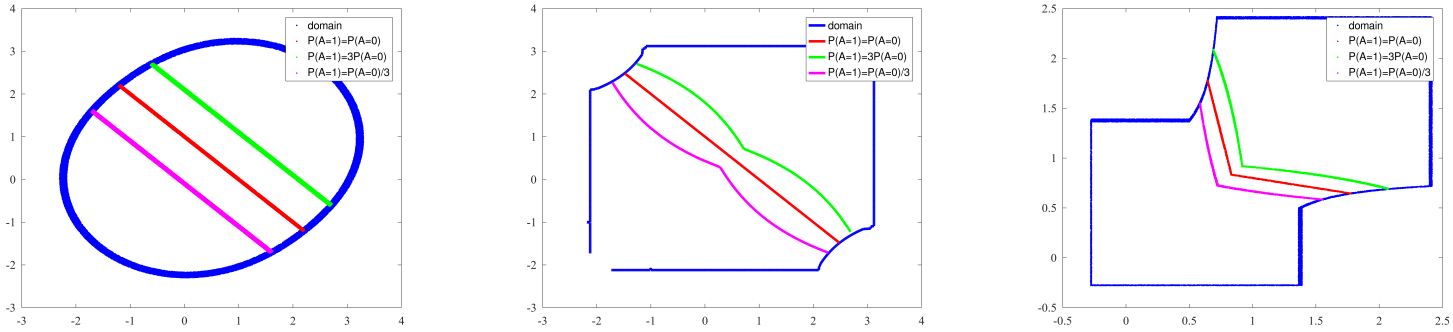


Figure 1: Locus of the relative probability of $\mathcal{A}=1$ to $\mathcal{A}=0$ for the three models described in equation (1): the first model is on the left, the second one in the middle and the third one on the right).

2 Experiments accounting for claim 1

2.1 Study of two correlated random variables

I am considering three models of correlated variables.

$$\left\{\begin{array}{l}\mathcal{X}_1=\mathcal{A}+\mathcal{E}\\ \mathcal{Y}_1=\mathcal{A}+\mathcal{F}\end{array}\right. \quad \left\{\begin{array}{l}\mathcal{X}_2=\mathcal{A}+3\mathcal{B}(\mathcal{C}-0.5)\\ \mathcal{Y}_2=\mathcal{A}+3\mathcal{B}(\mathcal{D}-0.5)\end{array}\right. \quad \left\{\begin{array}{l}\mathcal{X}_3=\mathcal{A}+\mathcal{B}+3\mathcal{B}(\mathcal{C}-0.5)\\ \mathcal{Y}_3=\mathcal{A}+\mathcal{B}+3\mathcal{B}(\mathcal{D}-0.5)\end{array}\right. \tag{1}$$

$$\text{with } \mathcal{A}\mapsto\mathcal{B}, \quad \mathcal{B},\mathcal{C},\mathcal{D}\mapsto\mathcal{U}(0,1) \text{ and } \mathcal{E},\mathcal{F}\mapsto\mathcal{N}(0,1)$$

All random variables $(\mathcal{A},\mathcal{B},\mathcal{C},\mathcal{D},\mathcal{E},\mathcal{F})$ are assumed independent.

- $\mathcal{B}$ is a discrete probability distribution with two evenly possible outputs 0 and 1, (also called Bernoulli trial).
- $\mathcal{N}(0,1)$ is the Gaussian centered and normalized distribution.
- $\mathcal{U}(0,1)$ is the Uniform distribution along $[0,1]$.

All three models intend to explain the gray-level of any two neighboring pixels. The first one addresses uniform noise, the second one, a spatially variable noise. And the third one addresses a spatially varying noise whose average value depends on its strength (for instance, the shadow of a cloud or a reflecting haze).

- The first model assumes additive white Gaussian noise with **known** standard deviation.
- The second model assumes additive white Gaussian noise with **unknown** standard deviation.
- The third model assumes additive **non-centered** white noise with **unknown** standard deviation.

These simulations are actually difficult to achieve, and to make them easier (availability of simple analytical formulas), I used, centered normalized uniform distributions in replacement of Gaussian distributions, this explains the values 3 and 0.5 that can be seen in equation (1), as these preserve mean and variance in the approximation.

Figure 1 shows, for the three models of equations (1), the likelihood of $\mathcal{A}=1$ knowing the values of $\mathcal{X}$ and $\mathcal{Y}$. Each point in each graph refers to specific values of $\mathcal{X}$ and $\mathcal{Y}$, $\mathcal{X}$ is on the left axis and $\mathcal{Y}$ is on the vertical axis. In figure 1, The blue contours enclose areas in the $\mathcal{X},\mathcal{Y}$-space with probability 0.95: most often drawn values of $\mathcal{X}$ and $\mathcal{Y}$ are within these areas. The red middle line separates the areas where $\mathcal{A}=1$ is more likely from those where $\mathcal{A}=0$ is more likely. The green and above line indicates where the probability of $\mathcal{A}=1$ is three time greater than the probability of $\mathcal{A}=0$. The magenta and below line indicates where the probability of $\mathcal{A}=0$ is three time greater than the probability of $\mathcal{A}=1$.

In the **left** of figure 1, the red line is straight and its position and slope shows that the theoretical estimator of $\mathcal{A}$ is the proximity of $\frac{\mathcal{X}_1+\mathcal{Y}_1}{2}$ with 1 vs 0. This estimator consists in averaging features of neighboring pairs of pixels.

In the **middle** of figure 1, the red line is the same (straight, centered and a (-1)-slope), it shows the same theoretical estimator of $\mathcal{A}$ consisting in averaging features (i.e. $\frac{\mathcal{X}_2+\mathcal{Y}_2}{2}$). The two other lines are not exactly straight

lines, their distance with the red line is smallest when $\mathcal{X}_2$ and $\mathcal{Y}_2$ are equal, it increases as $\mathcal{X}_2$ and $\mathcal{Y}_2$ become different and it eventually decreases. In this second model, $\mathcal{B}$ stands for the unknown standard deviation of a Gaussian noise. A low standard deviation is, on one hand, more likely when $\mathcal{X}_2$ and $\mathcal{Y}_2$ have similar values. On the other hand, a low standard deviation happens to be also more likely when $\mathcal{X}_2$ and $\mathcal{Y}_2$ have very different values. The latter is an unwanted side effect of approximating Gaussians with uniform laws, as when $\mathcal{X}_2$ and $\mathcal{Y}_2$ are very different, their sum is close to their maximum possible value and the reliability of the prediction of $\mathcal{A}$ is higher. Here a higher reliability of the sum as an estimator pushes the two lines closer to the red line.

In the **right** of figure 1, the red line is no longer a straight line, a rough estimation is $\min(\mathcal{X}_3, \mathcal{Y}_3)$. My interpretation is that the value of $|\mathcal{X}_3 - \mathcal{Y}_3|$ gives an estimation of $\mathcal{B}$ which now modifies the relationship between $\mathcal{A}$ and $\frac{\mathcal{X}_3+\mathcal{Y}_3}{2}$. I am suggesting the following understanding of what happens:

$$|\mathcal{X}_3 - \mathcal{Y}_3| \approx \mathcal{B}\mathrm{E}\left[\left|\mathcal{C} - \frac{1}{2}\right| + \left|\mathcal{D} - \frac{1}{2}\right|\right] = \frac{3}{2}\mathcal{B} \;\Rightarrow\; \mathcal{A} = \frac{\mathcal{X}_3 + \mathcal{Y}_3}{2} - \mathcal{B} \approx \frac{\mathcal{X}_3 + \mathcal{Y}_3}{2} - \frac{2}{3}|\mathcal{X}_3 - \mathcal{Y}_3| = -\frac{\mathcal{X}_3 + \mathcal{Y}_3}{6} + \frac{4}{3}\min(\mathcal{X}_3, \mathcal{Y}_3)$$

where E is the expectation operator.

The point I am making with these simulations is that these three models are generally considered as *linear* models and investigating them with the cross correlation is not helpful. For the three models, I got the following estimates.

$$\begin{cases} \mathrm{E}[\mathcal{X}_1] \approx 0.50 \\ \mathrm{E}[\mathcal{Y}_1] \approx 0.50 \\ \sqrt{\mathrm{var}[\mathcal{X}_1]} \approx 1.12 \\ \sqrt{\mathrm{var}[\mathcal{Y}_1]} \approx 1.12 \\ \frac{\mathrm{E}[(\mathcal{X}_1-0.5)(\mathcal{Y}_1-0.5)]}{\sqrt{\mathrm{var}[\mathcal{X}_1]}\sqrt{\mathrm{var}[\mathcal{Y}_1]}} \approx 0.20 \end{cases} \quad \begin{cases} \mathrm{E}[\mathcal{X}_2] \approx 0.50 \\ \mathrm{E}[\mathcal{Y}_2] \approx 0.50 \\ \sqrt{\mathrm{var}[\mathcal{X}_2]} \approx 0.71 \\ \sqrt{\mathrm{var}[\mathcal{Y}_2]} \approx 0.71 \\ \frac{\mathrm{E}[(\mathcal{X}_1-0.5)(\mathcal{Y}_1-0.5)]}{\sqrt{\mathrm{var}[\mathcal{X}_1]}\sqrt{\mathrm{var}[\mathcal{Y}_1]}} \approx 0.50 \end{cases} \quad \begin{cases} \mathrm{E}[\mathcal{X}_3] \approx 1 \\ \mathrm{E}[\mathcal{Y}_3] \approx 1 \\ \sqrt{\mathrm{var}[\mathcal{X}_3]} \approx 0.76 \\ \sqrt{\mathrm{var}[\mathcal{Y}_3]} \approx 0.76 \\ \frac{\mathrm{E}[(\mathcal{X}_3-1)(\mathcal{Y}_3-1)]}{\sqrt{\mathrm{var}[\mathcal{X}_3]}\sqrt{\mathrm{var}[\mathcal{Y}_3]}} \approx 0.57 \end{cases} \tag{2}$$

The thread I am following is to extract information and use it to improve the estimation process. Note that this can be regarded as trying to fit a nonlinear relationship and hence this somehow accounts for the use of machine learning and more specifically of neural networks applied on pairs of nearby pixels.

3 Description of the problem addressed

To exploit hyperspectral images and retrieve the information regarding land cover and land use, it is common practice to consider a supervised or semi-supervised classification problem wherein the class membership of a small/large number of pixels feeds a machine learning problem and yields a classification map. Many metrics are available to measure performance, they include Overall Accuracy (OA) and Average Accuracy (AA). Performance is also measured by looking at classification maps.

The labeled pixels form a training set and in a practical application, they should be available not only in a couple of reference images but in many others to account for the diversity of land covers and land uses and their relationship with the hyperspectral image. So supervised and semi-supervised classification problems are not reflecting the exact practical issues. To study my viewpoint, that is to show that more information can be extracted form a given hyperspectral image, I am considering the clustering problem described with the following definition and notations.

Definition 1. *Given a hyperspectral image* $\mathtt{I}$ *and a number of regions* G*, the objective is to find a clustered image* $\mathtt{F}$ *close to a ground truth clustered image* $\mathtt{G}$ *in the sense of high values of OA and AA, where*

- $\mathtt{I} = [\mathtt{I}_{mn}]$ *is a hyperspectral image.*
- m *is an index indicating location based on raster-scanning order.*
- n *is an index indicating the bandwidth.*
- $\mathtt{I}_{mn}$ *is the intensity of pixel located at* m *for bandwidth* n*.*
- $\mathcal{M}$ *is the set of all possible locations in* $\mathtt{I}$*, its size* $|\mathcal{M}|$ *is the number of pixels in* $\mathtt{I}$*.*
- $\mathtt{G} = [\mathtt{G}_m]$ *is the ground truth image that is assumed to be available for this specific image to evaluate the performance of the clustering technique.*
- $g = \mathtt{G}_m$ *is an integer indicating the class membership of pixel located at* m*.*

- *G is the number of classes, and the list of classes is $\mathcal{G} = \{1 \dots G\}$.*
- *$\mathtt{F} = [\mathtt{F}_m]$ is the clustered image that we are looking for.*
- *$f = \mathtt{F}_m$ is an integer indicating the cluster f to which the pixel located at m belongs to.*

Different from those related to classification, clustering images raises major practical issues. Given a hyperspectral image, one may expect to find very different kind of information: roadway mapping, crop mapping, building damage assessment, drought stress detecting. How could we expect a single technique yield very different maps. My intent is to focus on extracting more information from a given hyperspectral image, leaving as future research problem the task of adapting these techniques with contextual information when focusing on a specific kind of information.

We derive the clustering evaluation techniques from those evaluating classifications in the following section.

4 Adapting classification metrics and representations

Definitions of OA and AA for classification images can be seen as based on the confusion matrix $\mathtt{C} = [C_{gf}]$ defined as

$$\mathtt{C}_{gf} = |\{m \in \mathcal{M} \,|\mathtt{G}_m = g \text{ and } \mathtt{F}_m = f\}| \tag{3}$$

where $|\{\dots\}|$ counts the number of elements contained in a set $\{\dots\}$.

Note that for the sake of simplicity, I here omit to specify that $\mathtt{G}$ is the ground truth classified image and $\mathtt{F}$ is the tested classified function, (for more complete information, it should be written $\mathtt{C}_{gf}(\mathtt{G}, \mathtt{F})$).

$\mathtt{C}_{gf}$ can be computed using functions

$$\mathtt{C}_{gf} = \sum_{m \in \mathcal{M}} \mathbf{1}\left(\mathtt{G}_m = g\right) \mathbf{1}\left(\mathtt{F}_m = f\right) \tag{4}$$

where $\mathbf{1}(\dots)$ is 1 when the three dots are replaced by a true proposition an 0 when replaced by a false proposition.

$\mathtt{C}_{gf}$ can also be written with set theory

$$\mathtt{C}_{gf} = |\mathtt{G}^{-1}(\{g\}) \bigcap \mathtt{F}^{-1}(\{f\})| \tag{5}$$

where $\mathtt{G}^{-1}(\{g\}) = \{m \in \mathcal{M} | \mathtt{G}(m) = g\}$ is the set of pixels classified as g by $\mathtt{G}$ and $\mathtt{F}^{-1}(\{f\}) = \{m \in \mathcal{M} | \mathtt{F}(m) = f\}$ is the set of pixels classified as f by $\mathtt{F}$.

When $\mathtt{F}$ is a classified image, OA and AA are defined as

$$\mathrm{OA} = \frac{\sum\limits_{g=1}^{G} \mathtt{C}_{gg}}{\sum\limits_{g=1}^{G}\sum\limits_{f=1}^{F} \mathtt{C}_{gf}} \quad \mathrm{AA} = \frac{1}{G}\sum_{g=1}^{G} \frac{\mathtt{C}_{gg}}{\sum\limits_{f=1}^{F} \mathtt{C}_{gf}} \tag{6}$$

For more complete information, it should be indicated which classified image is being compared with which ground truth image as in $\mathrm{OA}(\mathtt{G}, \mathtt{F})$ and $\mathrm{AA}(\mathtt{G}, \mathtt{F})$).

Here are a few remarks on these expressions.

- The denominator of OA is the number of pixels:

$$\sum_{g=1}^{G}\sum_{f=1}^{F} \mathtt{C}_{gf} = |\mathcal{M}|$$

- The denominator of AA is the number of pixels of each class, that I denote c_g

$$c_g = \left|\mathtt{G}^{-1}(\{g\})\right| = \sum_{f=1}^{F} \mathtt{C}_{gf} \tag{7}$$

- The numerator of OA is the number of correctly classified pixels:

$$\sum_{g=1}^{G} \mathtt{C}_{gg} = \sum_{m \in \mathcal{M}} \mathbf{1}(\mathtt{G}(m) = \mathtt{F}(m)) = \left|(\mathtt{G} - \mathtt{F})^{-1}(\{0\})\right|$$

- OA is a symmetric operator, whereas AA is not exactly one because producer accuracy is different from user accuracy:

$$\mathrm{OA}(\mathtt{G}, \mathtt{F}) = \mathrm{OA}(\mathtt{F}, \mathtt{G}) \tag{8}$$

To adapt these definitions to clustering, my proposition is to estimate Φ a mapping of clusters into meaningful regions:

$$(\Phi \circ \mathtt{F})^{-1}(\{1\}) \dots (\Phi \circ \mathtt{F})^{-1}(\{G\}) \tag{9}$$

Φ is selected by maximizing the induced OA.

$$\phi(f) = \arg\max_{g \leq G} \mathtt{C}_{gf} \tag{10}$$

Thanks to ϕ, we get new definitions of $\mathrm{OA}_{\mathtt{C}}$ and $\mathrm{AA}_{\mathtt{C}}$.

$$\mathrm{OA}_{\mathtt{C}} = \frac{\sum_{f=1}^{F} \max_{g} \mathtt{C}_{gf}}{\sum_{g=1}^{G} \sum_{f=1}^{F} \mathtt{C}_{gf}} \quad \mathrm{AA}_{\mathtt{C}} = \frac{1}{G} \sum_{g=1}^{G} \frac{\sum_{f=1}^{F} \mathtt{C}_{gf} \mathbf{1}(g = \phi(f))}{\sum_{f=1}^{F} \mathtt{C}_{gf}} \tag{11}$$

$\mathrm{OA}_{\mathtt{C}}$ and $\mathrm{AA}_{\mathtt{C}}$ can also be expressed as classical metrics computed on a the confusion matrix obtained with $\Phi \circ \mathtt{F}$.

$$\mathrm{OA}_{\mathtt{C}} = \mathrm{OA}\left(\mathtt{G}, \Phi \circ \mathtt{F}\right) \quad \text{and} \quad \mathrm{AA}_{\mathtt{C}} = \mathrm{AA}\left(\mathtt{G}, \Phi \circ \mathtt{F}\right) \tag{12}$$

It should be noted, that $\mathrm{OA}_{\mathtt{C}}$ is no longer a symmetric operator because ϕ acts on the tested clustering map, adapting $\mathtt{F}$ to $\mathtt{G}$ not the other way around. ϕ is by design maximizing $\mathrm{OA}_{\mathtt{C}}$, presumably it is not maximizing $\mathrm{AA}_{\mathtt{C}}$. I am guessing that all classification metrics can be extended to clustering maps using ϕ as defined in equation (10).

We have three properties concerning $\mathrm{OA}_{\mathtt{C}}$.

- When $\mathtt{F}$ is one single cluster, $\mathrm{OA}_{\mathtt{C}}$ depends on the number of pixels in the greatest cluster of $\mathtt{G}$.

$$\mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathbf{1}_{\mathcal{M}}) = \frac{1}{|\mathcal{M}|} \max_{g} c_g$$

  where c_g is the number of pixels in class g according to $\mathtt{G}$: $c_g = |\mathtt{G}^{-1}(\{g\})|$. This is proved by lemma 1 (p. 26 in appendix).

- This value is actually a lower bound for any clustered image

$$\frac{1}{|\mathcal{M}|} \max_{g} c_g \leq \mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F})$$

  This is proved by lemma 2 (p. 27 in appendix): $\mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F}) \geq \mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathbf{1}_{\mathcal{F}} \circ \mathtt{F}) = \mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathbf{1}_{\mathcal{M}}) = \frac{1}{|\mathcal{M}|} \max_{g} c_g$

- When in a clustered image $\mathtt{F}$, a specific cluster f_0 is subdivided in two new clusters, (let us denote the new clustered image $\mathtt{F}'$), $\mathrm{OA}_{\mathtt{C}}$ is not decreased:

$$\mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F}') \geq \mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F})$$

  This is proved again with lemma 2

Proof. We assume that in $\mathtt{F}'$, part of cluster f_0 remains in cluster f_0 and part is assigned to cluster $F+1$, and that the $F-1$ remaining clusters are not modified:

$$\mathtt{F}'(m) = \begin{cases} \mathtt{F}(m) & \text{if} \quad \mathtt{F}(m) \neq f_0 \\ f_0 & \text{if} \quad \mathtt{F}'(m) = f_0 \\ F+1 & \text{if} \quad \mathtt{F}'(m) > F+1 \end{cases}$$

We then build a mapping of $\mathtt{F}'$ into $\mathtt{F}$:

$$\phi'(f') = \begin{cases} f' & \text{if} \quad f' \neq f_0 \text{ and } f' \leq F \\ f_0 & \text{if} \quad f' \in \{f_0, F+1\} \end{cases}$$

This choice of ϕ' ensures that $\mathtt{F} = \phi \circ \mathtt{F}'$ and by application of lemma 2 that $\mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F}') \geq \mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F})$ □

I am **conjecturing** that similar properties exist for $\mathrm{AA}_{\mathtt{C}}$

- When $\mathtt{F}$ is one single cluster, $\mathrm{AA}_{\mathtt{C}}$ depends on the number of classes in $\mathtt{G}$.

$$\mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathbf{1}_{\mathcal{M}}) = \frac{1}{G}$$

- This value is actually a lower bound for any clustered image

$$\frac{1}{G} \leq \mathrm{AA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F}) \leq 1$$

- When in a clustered image, a region is subdivided in two new regions, $\mathrm{AA}_{\mathtt{C}}$ is not decreased.

These two properties tell us that the range of values of $\mathrm{OA}_{\mathtt{C}}$ and $\mathrm{AA}_{\mathtt{C}}$ are reduced to values ranging respectively from $\frac{1}{|\mathcal{M}|}\max_g c_g$ and 1 and from $\frac{1}{G}$ and 1. To extend these range of values up to $[0,1]$, one might use the following linearly modified metrics:

$$\mathrm{OA}_{\mathtt{R}} = \frac{\mathrm{OA}_{\mathtt{C}} - \frac{1}{|\mathcal{M}|}\max_g c_g}{1 - \frac{1}{|\mathcal{M}|}\max_g c_g} \text{ and } \mathrm{AA}_{\mathtt{R}} = \frac{\mathrm{AA}_{\mathtt{C}} - \frac{1}{G}}{1 - \frac{1}{G}} \tag{13}$$

Definition 1 is now replaced by a more specific definition.

Definition 2.

$$\widehat{\mathtt{F}} = \underset{\mathtt{F} \in \mathcal{G}^{\mathcal{M}}}{\arg\max}\ OA_{\mathtt{C}}(\mathtt{G}, \mathtt{F})$$

where the optimization is done using only information read on $\mathtt{I}$ *and only clustered functions with no more than* G *clusters are considered.*

Classification maps is also used to evaluate classification techniques. I suggest using the following definition. Φ to display such maps.

Definition 3. *Let* $\mathtt{F}$ *be a clustering function and* $\mathtt{G}$ *be a ground truth map. Let* Φ *a labeling function of* $\mathtt{F}$ *defined by equation (10). The following function maps the image in* $\mathcal{G}$*, it is the definition of the clustering map: all pixels belonging to a value* g *in* $\mathcal{G}$ *are colored in a specific gray-level.*

$$\begin{array}{l} \mathcal{M} \to \mathcal{G} \\ m \mapsto \Phi \circ \mathtt{F}(m) \end{array} \tag{14}$$

Note that Φ uses information extracted from $\mathtt{G}$ and may merge several cluster when this increases the induced OA.

Table 1: Performances obtained with the five experiments described in sections 5 and 6.

name	section	CE	OA_c	AA_c	figure
$\mathcal{A}$	5	0.72	0.55	0.22	left of fig. 3
$\mathcal{B}$	5	0.02	0.68	0.45	right of fig. 4
$\mathcal{C}$	6	0.27	0.63	0.36	left of figure 5
$\mathcal{D}$	6	0.27	0.63	0.36	left of figure 6
$\mathcal{E}$	6	0.23	0.64	0.37	right of figure 6

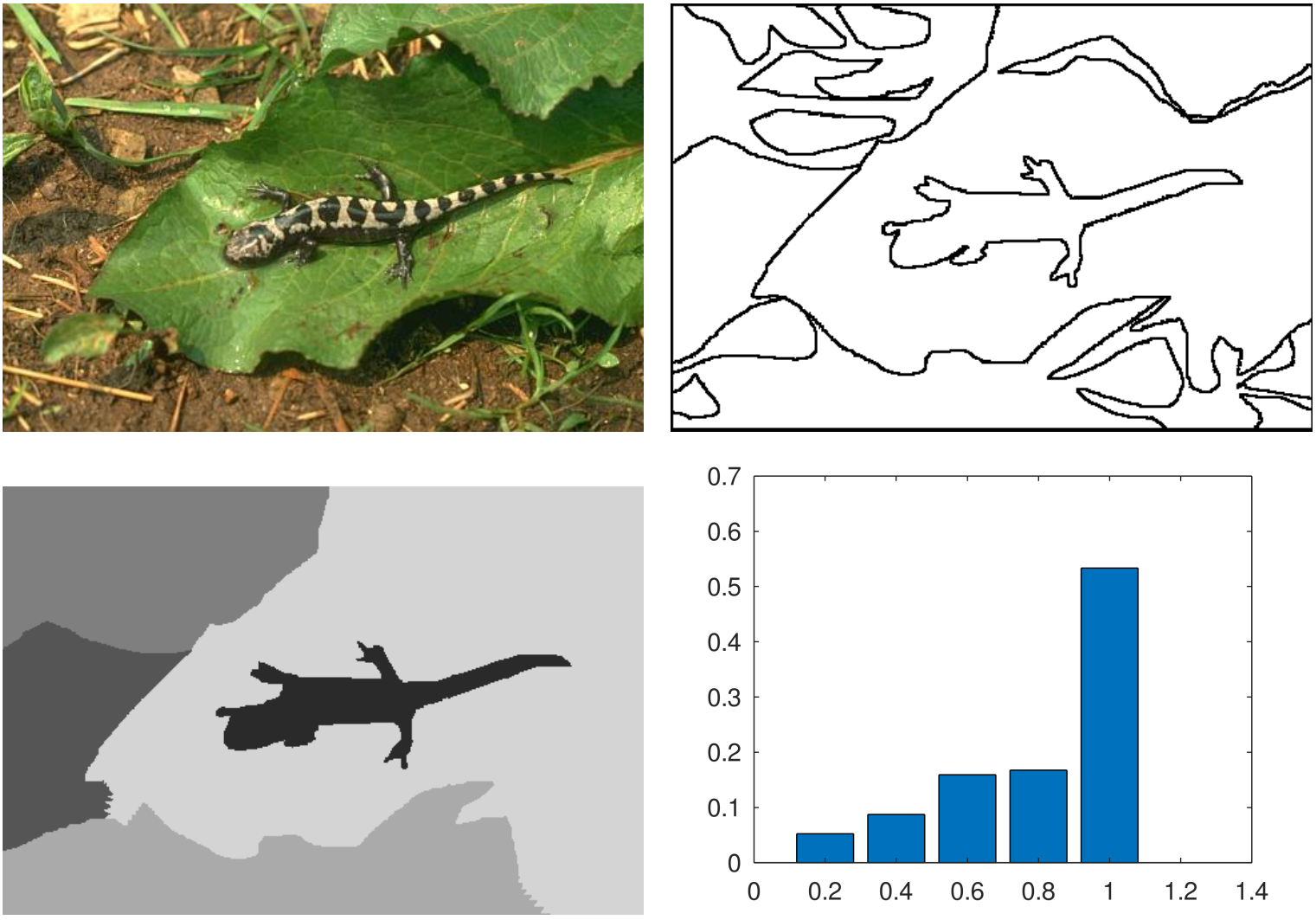


Figure 2: Left and above: the original RGB-image. Right and above: edge definition of the ground truth for clustering. Left and below: ground truth clustering map. Right and below: relative amount of pixels in each ground truth cluster.

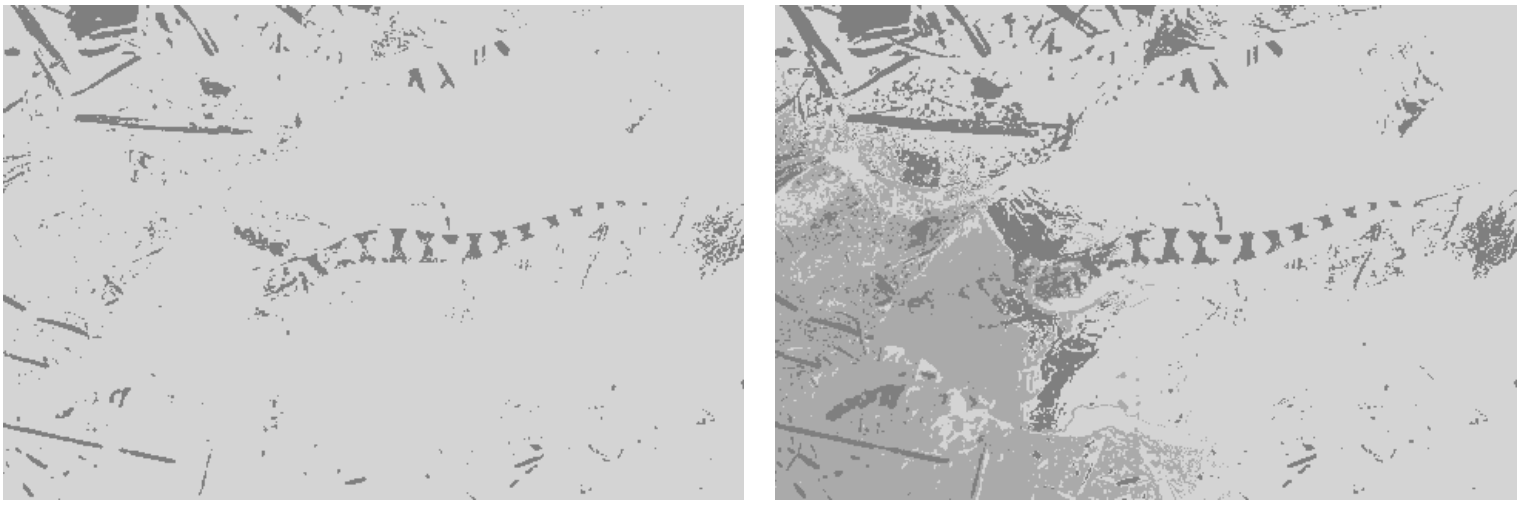


Figure 3: Left: clustering map of experiment $\mathcal{A}$ obtained with the k-means algorithm. Right: clustering map obtained also with k-means and with $\rho\mathcal{I}$ and $\rho\mathcal{J}$ as supplementary features. $\rho = 1$.

5 Experiments accounting for claim 2

Table 1 displays the results obtained throughout the five experiments described in sections 5 and 6.

For the sake of simplicity, I considered the objective of clustering an RGB-image, shown in the upper left of figure 2. A ground truth in terms of clustering is shown in the upper right of figure 2. This edge map is transformed into a clustering map shown in the lower left of figure 2, this is *mainly* done using `bwconncomp` in Octave, but it could also have been done using an algorithm called `edge2map` and detailed in appendix A.2. Actually, the ground truth edges define more than 5 regions, the algorithm uses the morphologic `open` with increasing sizes of structuring element to assign the remaining pixels to one of the 5 regions. Interestingly, this supplementary task seems to be equivalent to suppressing some edges of the upper right image. This supplementary task is described in `increase_clusters` in appendix A.2. Each region is filled with a gray color that is selected base on its ranking in terms of number of samples.

$$g_1 \leq g_2 \Leftrightarrow \left|\mathtt{G}^{-1}(\{g_1\})\right| \leq \left|\mathtt{G}^{-1}(\{g_2\})\right| \tag{15}$$

Note the very important spatial continuity shown here, it is indeed expected for any yielded clusters. We here assume that we know the number of clusters, here $G = 5$.

To properly address claim 2, it would have been more appropriate to test a Markov Random Field algorithm. Instead I chose to consider `K-means` as a basic clustering technique and to introduce some modifications to discuss claim 2.

A specificity of this image and of the provided ground truth is that clusters appear imbalanced. The most populated contains more than half of the samples. In the lower right graph of figure 2, a plot shows the relative amount of samples on the y-axis of each clusters: clusters are displayed on the x-axis as bars whose height indicates the relative amount of samples and whose location is the class index g.

The `K-means` algorithm, minimizes iteratively distances in the feature space among pixels sharing the same cluster. Its result, called experiment $\mathcal{A}$, is shown on the left of figure 3 using the methodology defined in definition 3. $\mathtt{C}(\mathtt{G},\mathtt{F})$ is computed with equation (3). More specifically, a first confusion matrix, $\mathtt{C}(\mathtt{G},\mathtt{F})$ is obtained considering $\mathtt{F}$ as a classical classification map, it is displayed after being normalized on the left of equation (16). This first confusion matrix is transformed into a more appropriate confusion matrix, $\mathtt{C}(\mathtt{G},\Phi\circ\mathtt{F})$, displayed after being normalized on the right of

equation (16) and obtained by moving and adding columns.

$$\frac{\mathtt{C}(\mathtt{G},\mathtt{F})}{|\mathcal{M}|} = \begin{bmatrix} 0.01 & 0.01 & 0.01 & 0.01 & 0.02 \\ 0.01 & 0.03 & 0.00 & 0.03 & 0.02 \\ 0.04 & 0.04 & 0.02 & 0.05 & 0.01 \\ 0.02 & 0.05 & 0.01 & 0.05 & 0.04 \\ 0.10 & 0.14 & 0.01 & 0.19 & 0.10 \end{bmatrix} \quad \begin{matrix} \\ \\ \\ \\ \\ \mathrm{OA} = 0.20 \end{matrix} \quad \xrightarrow{\Phi} \quad \frac{\mathtt{C}(\mathtt{G},\Phi\circ\mathtt{F})}{|\mathcal{M}|} = \begin{bmatrix} 0.00 & 0.00 & 0.01 & 0.00 & 0.04 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.08 \\ 0.00 & 0.00 & 0.02 & 0.00 & 0.14 \\ 0.00 & 0.00 & 0.01 & 0.00 & 0.16 \\ 0.00 & 0.00 & 0.01 & 0.00 & 0.53 \end{bmatrix} \quad \begin{matrix} \\ \\ \\ \\ \\ \mathrm{OA}_{\mathtt{C}} = 0.55 \quad \mathrm{AA}_{\mathtt{C}} = 0.22 \end{matrix} \tag{16}$$

with Φ defined as

$$\Phi(1) = 5 \quad \Phi(2) = 4 \quad \Phi(3) = 3 \quad \Phi(4) = 5 \quad \Phi(5) = 5 \tag{17}$$

meaning that:

- the fifth column of the new confusion matrix is obtained by adding the fifth, the fourth and the first column of the older matrix;
- the second and the third columns of the old one have just been moved.

It is striking to see that the yielded clusters have little spatial continuity. Note that results are slightly modified each time the experiment is redone.

A way of introducing spatial continuity is to introduce the spatial coordinates as two new features denoted as $\rho\mathcal{I}$ and $\rho\mathcal{J}$, where $\rho > 0$ is a parameter accounting for the importance of the new features and $\mathcal{I}, \mathcal{J}$ are the centered and normalized coordinates, (note that for the sake of clarity, the other features had also been centered and normalized). With this normalization, $\mathcal{I}$ and $\mathcal{J}$ can be seen as some kind of random variable following a uniform law on the interval $[-\sqrt{3}, \sqrt{3}]$.

Right of figure 3 shows the clustering map obtained with $\rho = 1$ with a small increase in the $\mathrm{OA}_{\mathtt{C}}$ as compared to the left of figure 3. As with the previous experiment, the left confusion matrix is transformed thanks to a relabeling Φ-function into the right confusion matrix, with which $\mathrm{OA}_{\mathtt{C}}$ and $\mathrm{AA}_{\mathtt{C}}$ are computed.

$$\frac{\mathtt{C}(\mathtt{G},\mathtt{F})}{|\mathcal{M}|} = \begin{bmatrix} 0.03 & 0.01 & 0.00 & 0.00 & 0.00 \\ 0.01 & 0.01 & 0.06 & 0.00 & 0.00 \\ 0.02 & 0.04 & 0.01 & 0.00 & 0.09 \\ 0.04 & 0.01 & 0.07 & 0.04 & 0.00 \\ 0.17 & 0.04 & 0.05 & 0.16 & 0.11 \end{bmatrix} \quad \begin{matrix} \\ \\ \\ \\ \\ \mathrm{OA} = 0.20 \end{matrix} \quad \xrightarrow{\Phi} \quad \frac{\mathtt{C}(\mathtt{G},\Phi\circ\mathtt{F})}{|\mathcal{M}|} = \begin{bmatrix} 0.00 & 0.00 & 0.01 & 0.00 & 0.03 \\ 0.00 & 0.00 & 0.01 & 0.06 & 0.01 \\ 0.00 & 0.00 & 0.04 & 0.01 & 0.11 \\ 0.00 & 0.00 & 0.01 & 0.07 & 0.08 \\ 0.00 & 0.00 & 0.04 & 0.05 & 0.44 \end{bmatrix} \quad \begin{matrix} \\ \\ \\ \\ \\ \mathrm{OA}_{\mathtt{C}} = 0.56 \quad \mathrm{AA}_{\mathtt{C}} = 0.31 \end{matrix}$$

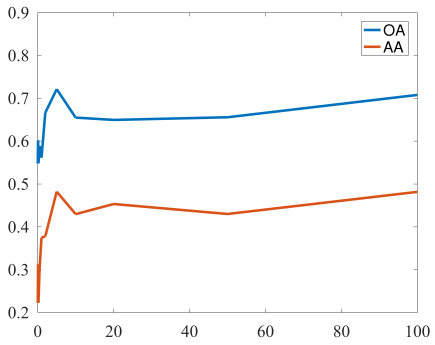
with

$$\Phi(1) = 5 \quad \Phi(2) = 3 \quad \Phi(3) = 4 \quad \Phi(4) = 5 \quad \Phi(5) = 5$$

Left of figure 4 shows the evolution of $\mathrm{OA}_{\mathtt{C}}$ and $\mathrm{AA}_{\mathtt{C}}$ with respect to ρ. Both metrics tend to increase up to a certain value of ρ and they then either decrease or reach a plateau. The $\mathrm{OA}_{\mathtt{C}}$ never exceeds 0.7. Right of figure 4 shows the clustering map obtained with $\rho = 5$, a value that maximizes $\mathrm{OA}_{\mathtt{C}}$ in the left figure; this is called experiment $\mathcal{B}$. Unfortunately, the lack of resemblance with the ground truth of this good $\mathrm{OA}_{\mathtt{C}}$-performance only show that $\mathrm{OA}_{\mathtt{C}}$ and $\mathrm{AA}_{\mathtt{C}}$ seem not to be reliable metrics when they are below 0.7. Center of figure 4 shows the cross-correlation of the norm of the features as a function of ρ. It appears as a ρ-increasing function reaching very high values when ρ exceeds 5. I think this shows at the same time that adding new coordinates yields clusters that have good spatial continuity and makes it more difficult to drive the algorithm in a good direction.

6 An iterative clustering algorithm accounting for claims 3 and 4

The experiment $\mathcal{C}$ bears resemblance with the previous one, combining a metric (something similar to the cross-correlation of neighboring pixels) and an action modifying the feature space (similar to the adding of new coordinates).



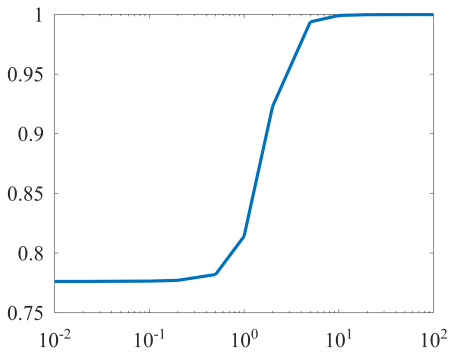


Figure 4: Left: $\mathrm{OA}_{\mathtt{c}}$ and $\mathrm{AA}_{\mathtt{c}}$ with respect to ρ. Middle: evolution of the cross-correlation of the feature norm of two neighboring pixels. Right: clustering map of experiment $\mathcal{B}$ with k-means and $\rho\mathcal{I}$ and $\rho\mathcal{J}$ as supplementary features with $\rho = 5$: $\mathrm{OA}_{\mathtt{c}} = 0.68$ and $\mathrm{AA}_{\mathtt{c}} = 0.45$.

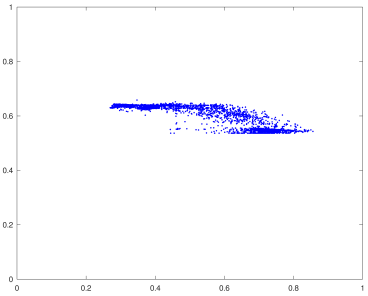
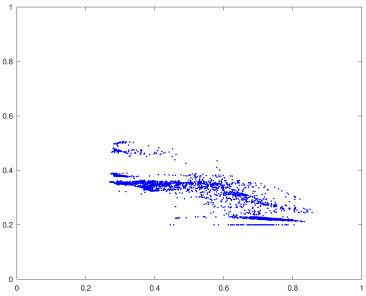




Figure 5: Left: clustering map of experiment $\mathcal{C}$ with $\mathrm{OA}_{\mathtt{c}} = 0.63$, $\mathrm{AA}_{\mathtt{c}} = 0.36$; middle and right: evolutions of $\mathrm{OA}_{\mathtt{c}}$ and $\mathrm{AA}_{\mathtt{c}}$ with respect to the CE-metric values.

Algorithm 1 EXPERIMENT

Require: `I` the image, and G the number of clusters.

Ensure: `F`.

Compute D the matrix containing the `I`-extracted features.
Draw P a 3×3 matrix.
Set $P_{\text{best}} := P$ and $J_{\text{best}} := +\infty$
for $t = 1 : 500$ **do**
    Compute `F` the clustering function using `kmeans` applied on DP.
    if $\mathrm{CE}(\mathtt{F}) < J_{\text{best}}$ **then**
        $P_{\text{best}} := P$ and $J_{\text{best}} := \mathrm{CE}(\mathtt{F})$
    Draw P taking into account P_{best}.
Compute `F` the clustering function using `kmeans` applied on DP_{best}.

The key idea is that there should be no relationship between the metric and the modifying action other than that of finding a relevant clustering. I think that information can be extracted by an intensive exchanging information between the metric and the modifying action.

The metric addresses clustering maps, it counts the number of pixels on edges, such pixels are defined as those for which there exists a neighboring pixel belonging to a different cluster.

$$\mathrm{CE}(\mathtt{F}) = \sum_{m=1}^{M} \mathbf{1}\left(\exists m' \in \mathtt{N}(m),\ \mathtt{F}_m \neq \mathtt{F}_{m'}\right) \tag{18}$$

where $\mathtt{N}(m)$ is the set of the eight points surrounding pixel m, and $\exists$ means that there exists pixels m' in $\mathtt{N}(m)$. CE is here implemented in Octave, its code is listed in section C.2. Note that when CE is applied to the clustering images obtained in experiments $\mathcal{A}$ and $\mathcal{B}$ shown left of figure 3 and right of figure 4, it reads respectively 0.72 and 0.02. This is consistent with a high increase in spatial continuity.

The basic algorithm is to draw randomly a very large number of linear mappings and to consider the clustering map yielding the smallest value. It is briefly described in algorithm 1.

Steps 5 and 9 of algorithm 1 both refer to the product DP. This implements the application of a linear mapping, denoted P being a 3×3-matrix. It is applied on the feature space composed of the three features R, G, B, and it defines three new features F_1, F_2, F_3.

$$\begin{bmatrix} F_1 \\ F_2 \\ F_3 \end{bmatrix} = \begin{bmatrix} R \\ G \\ B \end{bmatrix} P \tag{19}$$

At each repeated step of the `for`-loop, each component of P is drawn independently.

$$P_{ij} \mapsto \mathcal{N}(0,1) \tag{20}$$

Step 8 in this algorithm is actually more complex. To increase the speed of the algorithm and to mimic the simulated annealing algorithm, P is be obtained by adding to the best P matrix, a smaller randomly drawn 3×3-matrix (with standard deviation of 0.1, 0.01 or 0.001, or by modifying one component, one column or one line). These actions are randomly chosen.

Steps 5 and 9 in this algorithm use the k-`means` algorithm to get clusters and equation (44) to transform these clusters into `F`. Note that this algorithm using a random walk to search for P_{best} requires that everything depending on each matrix P be fully deterministic. To this end, k-means is used in a special way displayed in appendix in section C.3.

The experiment shows that CE started at 0.72 and went down to 0.27. The final matrix P_{best} is :

$$P_{\mathrm{best}} = \begin{bmatrix} -0.00 & 0.15 & 0.18 \\ 0.08 & -0.12 & 0.07 \\ -0.09 & 0.04 & -0.26 \end{bmatrix}$$

The normalized confusion matrix is

$$\frac{\mathtt{C(G,F)}}{|\mathcal{M}|} = \begin{bmatrix} 0.01 & 0.00 & 0.00 & 0.00 & 0.04 \\ 0.05 & 0.01 & 0.00 & 0.00 & 0.03 \\ 0.08 & 0.03 & 0.01 & 0.02 & 0.02 \\ 0.11 & 0.01 & 0.00 & 0.00 & 0.04 \\ 0.02 & 0.04 & 0.14 & 0.30 & 0.04 \end{bmatrix}$$

$$\mathrm{OA} = 0.06$$

Because clusters are assigned with respect to the ground truth, we get:

$$\Phi(1) = 4 \quad \Phi(2) = 5 \quad \Phi(3) = 5 \quad \Phi(4) = 5 \quad \Phi(5) = 4$$

This means:

- the second, third and fourth columns are added to form the fifth column;

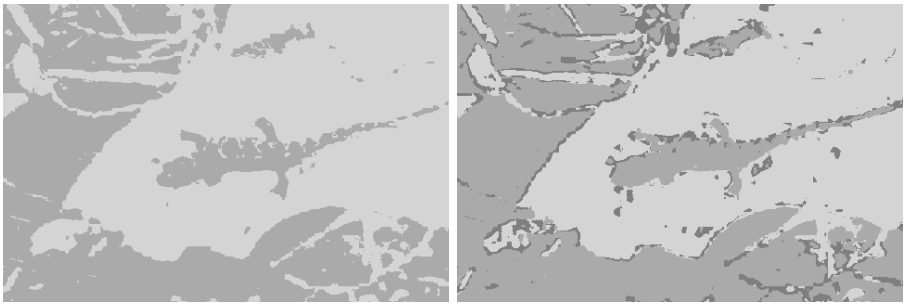


Figure 6: Left: clustering map of experiment $\mathcal{D}$ with $\mathrm{OA}_{\mathtt{C}} = 0.63$, $\mathrm{AA}_{\mathtt{C}} = 0.36$; Right: clustering map of experiment $\mathcal{E}$ with $\mathrm{OA}_{\mathtt{C}} = 0.64$, $\mathrm{AA}_{\mathtt{C}} = 0.37$.

- the first and fifth columns are added to form the fourth column;
- the first, second and third columns do not contain any pixels.

With these assignments, the new confusion matrix becomes:

$$\frac{\mathtt{C}(\mathtt{G},\Phi\circ\mathtt{F})}{|\mathcal{M}|} = \begin{bmatrix} 0.00 & 0.00 & 0.00 & 0.04 & 0.01 \\ 0.00 & 0.00 & 0.00 & 0.08 & 0.01 \\ 0.00 & 0.00 & 0.00 & 0.10 & 0.06 \\ 0.00 & 0.00 & 0.00 & 0.15 & 0.02 \\ 0.00 & 0.00 & 0.00 & 0.05 & 0.48 \end{bmatrix}$$
$$\mathrm{OA}_{\mathtt{C}} = 0.63 \quad \mathrm{AA}_{\mathtt{C}} = 0.36$$

Figure 5 shows on the left the clusters obtained as defined by $\Phi \circ \mathtt{F}^{-1}(\{c\})$. The graphs on the middle and on the right show the relationship between CE (on the horizontal axis) and first $\mathrm{OA}_{\mathtt{C}}$ then $\mathrm{AA}_{\mathtt{C}}$ (on the vertical axis). Clearly these are not decreasing deterministic curves, meaning that for a single value of CE, there can be better or worse values of $\mathrm{OA}_{\mathtt{C}}$ and $\mathrm{AA}_{\mathtt{C}}$. There appears a tendency to yield better values for $\mathrm{OA}_{\mathtt{C}}$ and $\mathrm{AA}_{\mathtt{C}}$ when CE is decreased. This is a key-property, to think about when selecting or designing a metric.

In experiment $\mathcal{C}$, we can see that $\mathrm{OA}_{\mathtt{C}}$ is below to that of experiment $\mathcal{B}$ and that the corresponding clustering maps shown in the left of figure 5 for $\mathcal{C}$ and in figure 4 for $\mathcal{B}$ are very different. Hence it appears that $\mathrm{OA}_{\mathtt{C}}$ is not a reliable evaluation metric (at least below 0.7). We here witness only a tendency when $\mathrm{OA}_{\mathtt{C}}$ is higher, it is more likely to have a better clustering image.

I made two other experiments called experiments $\mathcal{D}$ and $\mathcal{E}$.

Experiment $\mathcal{D}$ uses a fourth feature averaging the red, green, and blue components of the pixel located at the right, (it is the left pixel if no right neighbor is available). Similarly to the previous experiment, we get a matrix P_{best} of size 4×3 to transform four features into three features, to which is applied the k-`means` algorithm.

$$P_{\text{best}} = \begin{bmatrix} 0.24 & 0.06 & 0.03 \\ -0.03 & -0.13 & 0.06 \\ -0.19 & 0.13 & -0.10 \\ -0.01 & 0.01 & 0.01 \end{bmatrix}$$

We get a Φ-transformed confusion matrix et two evaluation metrics,

$$\frac{\mathtt{C}(\mathtt{G},\Phi\circ\mathtt{F})}{|\mathcal{M}|} = \begin{bmatrix} 0.00 & 0.00 & 0.00 & 0.04 & 0.01 \\ 0.00 & 0.00 & 0.00 & 0.08 & 0.01 \\ 0.00 & 0.00 & 0.00 & 0.10 & 0.06 \\ 0.00 & 0.00 & 0.00 & 0.15 & 0.02 \\ 0.00 & 0.00 & 0.00 & 0.05 & 0.48 \end{bmatrix}$$
$$\mathrm{OA}_{\mathtt{C}} = 0.63 \quad \mathrm{AA}_{\mathtt{C}} = 0.36$$

using the following Φ mapping.

$$\Phi(1) = 4 \quad \Phi(2) = 5 \quad \Phi(3) = 5 \quad \Phi(4) = 5 \quad \Phi(5) = 4$$

Although a greater diversity is offered to choose P, the algorithm seems not to have used this freedom and the results of $\mathcal{D}$ are very similar to those of $\mathcal{C}$. The last line of $P_{\mathcal{D}}$ shows that this added feature has not been used. The fact that the three first lines of $P_{\mathcal{D}}$ are different from those of $P_{\mathcal{C}}$ shows that there could be local minima and that exploring the searching space appears a difficult issue.

Experiment $\mathcal{E}$ uses the same features as experiment $\mathcal{C}$. It is different from $\mathcal{C}$ because the k-`means` algorithm is used to find 7 clusters instead of 5 and these 7 clusters are merged into 5 clusters using a very brute-force technique consisting in testing all possible mapping and considering the mapping which yields the minimal value for CE. This bears some resemblance with the proposal in the following section. To reduce the numerical complexity an algorithm removes the mapping yielding clusters that would be similar to other clusters up to some relabeling. Appendix C.4 lists the Octave code yielding the mapping sent to CE, an algorithm that can be seen as a simplified implementation of a merging operator explained in section 7.8. The good news is that this algorithm has been able to reduce CE down to 0.23. The $\mathrm{OA}_{\mathtt{C}}$- and $\mathrm{AA}_{\mathtt{C}}$-performances are slightly higher but with a so small difference, that little confidence should put in this increase, all the more as experiment $\mathcal{B}$ yields much higher $\mathrm{OA}_{\mathtt{C}}$- and $\mathrm{AA}_{\mathtt{C}}$-performances.

$$P_{\text{best}} = \begin{bmatrix} 0.16 & -0.01 & -0.02 \\ 0.03 & 0.12 & 0.12 \\ -0.20 & -0.17 & -0.15 \end{bmatrix}$$

$$\frac{\mathtt{C}(\mathtt{G},\Phi\circ\mathtt{F})}{|\mathcal{M}|} = \begin{bmatrix} 0.00 & 0.00 & 0.00 & 0.04 & 0.01 \\ 0.00 & 0.00 & 0.01 & 0.07 & 0.01 \\ 0.00 & 0.00 & 0.02 & 0.08 & 0.06 \\ 0.00 & 0.00 & 0.01 & 0.13 & 0.02 \\ 0.00 & 0.00 & 0.01 & 0.04 & 0.49 \end{bmatrix}$$

$$\mathrm{OA}_{\mathtt{C}} = 0.64 \quad \mathrm{AA}_{\mathtt{C}} = 0.37$$

$$\Phi(1) = 3 \quad \Phi(2) = 4 \quad \Phi(3) = 5 \quad \Phi(4) = 5 \quad \Phi(5) = 5$$

7 Description of the proposal

In this section and in the following sections, I am going to expose a broad family of proposed techniques instead a very specific one, for the following reasons.

- Let's assume that a proposed technique achieves some significant increase in performances. The field being new (or new to me), these performances are not indicative of which tasks are required and which are not required. A simple and performing experiment is undoubtedly more convincing and hence more useful for research practitioners.
- It is not satisfactory to compare performances obtained with clustering with those obtained with semi-supervised classification. To convince that there is a significant increase in performance, one might have to boost the technique.
- I cannot anticipate what difficulties will be met: numerical complexity, programming time, existence/amount of irrelevant clusters that have good spatial continuity? It may prove useful to adapt the technique to overcome the encountered difficulties while remaining consistent with the primary goals.

It is of great importance when designing a technique in line with the following proposal to focus on the following issues.

- Is the diversity of the space-feature operators sufficient to make the metric significantly decrease?

- Is this increase in diversity not changing the relationship between the metric and the performances of the clustering technique? (This would imply that a significant reduction of the metric value would not yield an increase in performances).
- Is the increase numeric complexity jeopardizing the possibility of testing this technique (i.e. of finding the optimal parameter values)?

In compliance with these claims, I am proposing to use feature-space operators defined in section 7.1. They are being used in three basic layouts (section 7.2, 7.3, 7.4). These layouts make use of four different operators described in the remaining sections (Split in section 7.5, Rotate in section 7.7, Merge in section 7.8 and Pooling in section 7.9).

7.1 Feature-space operator

We consider here notations describing operators in the feature space.

- $\mathtt{I}$ is a hyperspectral image regarded as a matrix whose rows are features arrays $\mathbf{I}_m = [\mathtt{I}(m,1)\dots\mathtt{I}(m,N)]$.
- $\mathtt{S}$ maps feature vectors in binary digits, thereby defining a binary space partitioning.
- $\overline{\mathtt{S}}$ is the complementary feature-space operator assigning the cluster 1 or 0 when $\mathtt{S}$ assigns 0 or 1.

$$\overline{\mathtt{S}}(\mathbf{I}_m) = 1 - \mathtt{S}(\mathbf{I}_m)$$

- CL combine different feature-space operators to assign a cluster to each feature array or equivalently yielding a clustering function $\mathtt{F}$ for each image $\mathtt{I}$.

$$\mathtt{F}(m) = \mathrm{CL}\left(\mathtt{S}_1\dots\right)(\mathbf{I}_m) \tag{21}$$

Definition 4. $\mathtt{F}$ *is a feature-space operator if and only if there exists an operator* $\mathtt{S}$ *such that*

$$m \in \mathcal{M} \Rightarrow \mathtt{F}(m) = \mathtt{S}(\mathbf{I}_m)$$

Such operators are invariant to a change of location in that a shifted image yields a shifted clustering map.

7.2 Ladder structure

The ladder structure shown in figure 7 for $G = 4$ is inspired by the `else if` instruction that enable chaining branch instructions. It is very similar to tree-classifiers. The feature space at the top level is the set of spectral arrays $\mathbf{I}_m = \{[\mathtt{I}(m,1)\dots\mathtt{I}(m,N)]\,|m \in \mathcal{M}\}$. Below the first Splitting operator S_1 sorts $\mathbf{I}_m$ in two categories, one corresponding to the first cluster $\mathtt{F}^{-1}(\{1\})$, and the second feeding S_2. This process is done in the same way $G-1$ times (i.e. three times in the figure). This technique yields a clustering function mapping pixel locations $m \in \mathcal{M}$ into G clusters $\mathcal{G}$ by having access to the pixel location of each spectral array.

On the example shown in figure 7, we have

$$\mathrm{CL}\left(\mathrm{S}_1,\mathrm{S}_2,\mathrm{S}_3,\mathrm{S}_4\right) := \left(\mathrm{S}_1 + 2\mathrm{S}_2\overline{\mathrm{S}}_1 + 3\mathrm{S}_3\overline{\mathrm{S}}_2\overline{\mathrm{S}}_1 + 4\mathrm{S}_4\overline{\mathrm{S}}_3\overline{\mathrm{S}}_2\overline{\mathrm{S}}_1\right) \tag{22}$$

An interesting feature of this structure is that the splitting operators are more decoupled than in the tree structure. Presumably, it could be less needed to iteratively improve each splitting criterion. On the other hand, one might have to adapt the spatial metrics to enable splitting operators yield imbalanced outputs (i.e. the number of pixels succeeding at S_1 is expected to be much smaller than those failing). Performance could be improved by considering a higher number of clusters (higher than G) and applying a Merging operator (of course not using the clustering map $\mathtt{G}$).

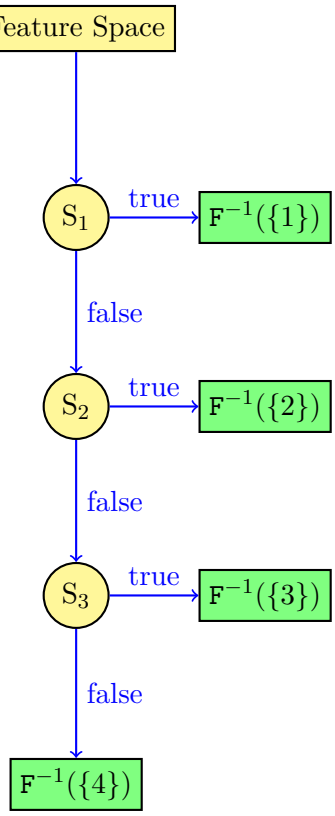


Figure 7: Ladder structure. There are three split operators S_f acting in the feature space (yellow), with for each, either a true output or a false output. All these operators yield in the data space (green), four clusters $\mathtt{F}^{-1}(\{f\})$.

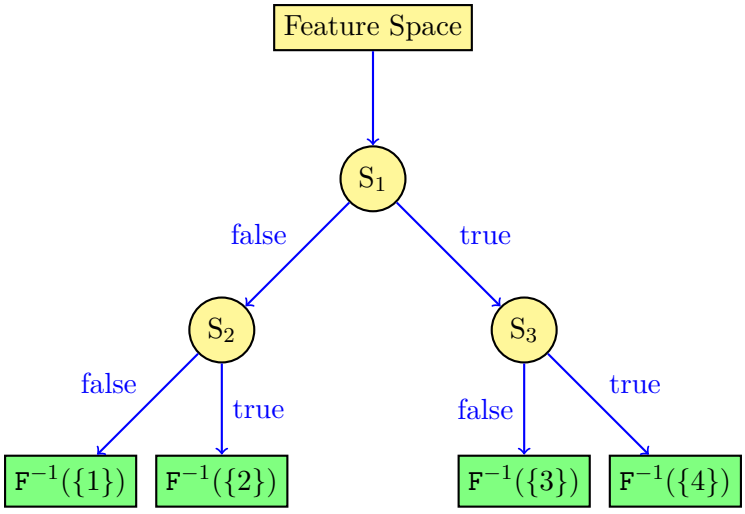


Figure 8: Tree structure. There are three split operators S_f with for each, either a true output or a false output, all these operators yield four clusters $\mathtt{F}^{-1}(\{f\})$.

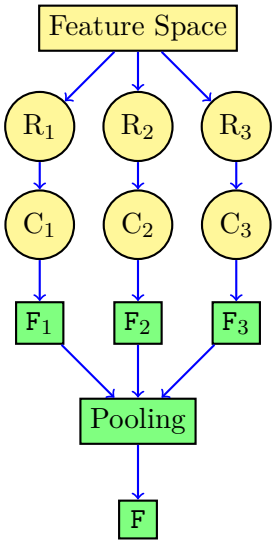


Figure 9: Forest structure. There are three clustering operators C_t transforming rotated feature $\mathrm{R}_t \circ \mathtt{I}$ into three clustering functions $\mathtt{F}_t$ with $t \in \{1, 2, 3\}$. These functions are pooled into a single clustering function $\mathtt{F}$.

7.3 Tree structure

The tree structure shown in figure 8 for $G = 4$ is similar to the tree structure used for classifiers. One difference lies in how each splitting criterion is selected, and because it uses a global spatial metric, the connections between the different splitting operators is an important issue. I am expecting the need when optimizing the parameters, to consider iteratively each splitting operators taking into account the choices made for the other splitting operators. Another difference lies in that increasing the depth of a tree increases the number of clusters, presumably beyond G. Note that for classifiers this is not an issue because thanks to the training set, leaves containing mostly or only samples of the same class are merged. Here I am expecting the need of the Merging operator to select the fusion of some clusters, it is described in section 7.8.

On the example shown in figure 8, we have

$$\mathrm{CL}\left(\mathrm{S}_1, \mathrm{S}_2, \mathrm{S}_3, \mathrm{S}_4\right) := \left(\overline{\mathrm{S}}_2\overline{\mathrm{S}}_1 + 2\mathrm{S}_2\overline{\mathrm{S}}_1 + 3\overline{\mathrm{S}}_3\mathrm{S}_1 + 4\mathrm{S}_3\mathrm{S}_1\right) \tag{23}$$

7.4 Forest structure

The forest structure shown in figure 9 where $T = 3$ clustering operators are grown independently similarly to the forest structure used for Random Forest classifiers. A first difference is in how diversity among trees is introduced: bagging is here not an easy option because spatial metrics cannot easily adapt when pixels are considered twice or when some pixels are missing. Instead I suggest using a Rotating operator. A second difference is in many ensemble classifiers, pooling consists in majority voting. Because trees are grown with different feature spaces, it is difficult to know how to label clusters of different trees. I suggest using the Pooling operator to overcome this issue.

7.5 Split operators

Split operators achieve two tasks, that of sorting pixels using a parameter-dependent criterion and that of finding the optimal parameter values optimizing a given metric $\mathtt{MT}$ for a specific splitting operator $\mathtt{F}_{\mathrm{SPLIT}}$ or for a clustering function yielded by the whole structure, (in classification, we would say prediction and training).

7.5.1 Split operators are decision stumps

- $\gtrless_o$ defines a parameter-dependent inequality operator

$$x \gtrless_o y \quad \Leftrightarrow \quad \begin{cases} x < y & \text{if} \quad o = 1 \\ x > y & \text{if} \quad o = -1 \end{cases} \tag{24}$$

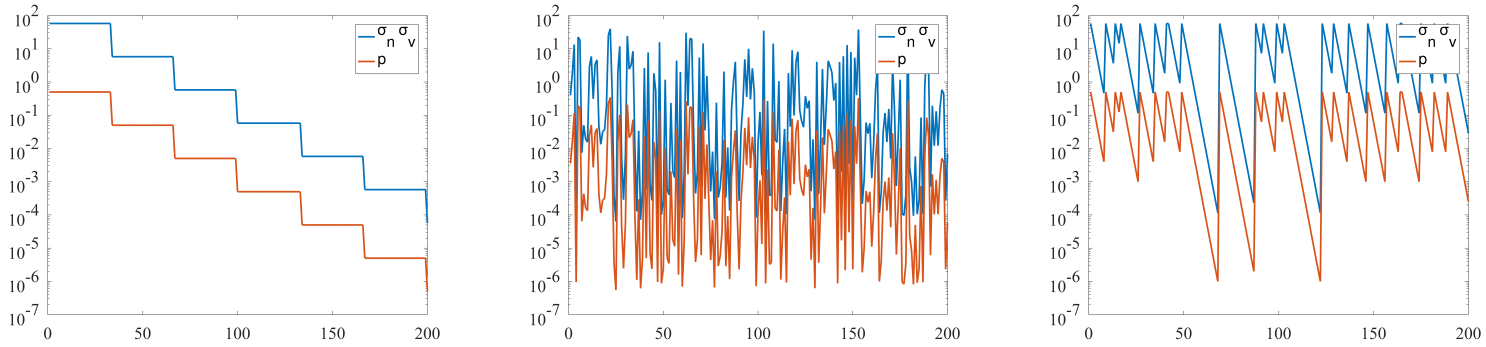


Figure 10: Values of σ_n, σ_v and p along a randomly simulated sequence of experiments. Left: classical simulated annealing, Middle: simplified simulated annealing used in algorithm 1. Right: proposed modified simulated annealing.

- $\mathtt{S}_{\mathrm{SPLIT}}$ is a decision stump comparing the value addressed by a single feature-index n with a threshold value v, where the comparison order depends on o.

$$\forall m \in \mathcal{M}, \quad \mathtt{S}_{\mathrm{SPLIT}}(m) := \mathbf{1}(I(m,n) \gtrless_o v) \quad = \begin{cases} \mathbf{1}(I(m,n) < v) & \text{if } o = 1 \\ \mathbf{1}(I(m,n) > v) & \text{if } o = -1 \end{cases} \tag{25}$$

  $\mathtt{S}_{\mathrm{SPLIT}}$ is also regarded as a Split operator assigning a cluster 0 or 1 to each feature array or equivalently yielding a binary space partitioning $\mathtt{F}_{\mathrm{SPLIT}}$.

$$m \in \mathcal{M} \Rightarrow \mathtt{F}_{\mathrm{SPLIT}}(m) = \mathtt{S}(\mathbf{I}_m) \tag{26}$$

7.6 Training of split operators

Algorithm 2 SPLIT

Require: CLK, MT
Ensure: n and v
1: Set $[n, v, o] := \left[\frac{N}{2}, \frac{1}{2}, 1\right]$, $[\sigma_n, \sigma_v, p_o] := \left[\frac{N}{2\sqrt{3}}, \frac{N}{2\sqrt{3}}, \frac{1}{2}\right]$, $J := -\infty$ and $[k_t, k_p] = [0, 20]$
2: **while** time $\leq$ CLK **do**
3: Draw $[n', v', o']$ according to equation (29)
4: Update $[\sigma_n, \sigma_v, p_o]$ according to equations (31), (32), (33).
5: Set $\mathtt{F}_{\mathrm{SPLIT}}$ according to equation (25) or F if a whole structure is to be globally trained.
6: Evaluate $J' := \mathtt{MT}(\mathtt{F}_{\mathrm{SPLIT}})$ or $J' := \mathtt{MT}(\mathtt{F})$ according to (18) or to section 8.
7: **if** $J' < J$ **then**
8: Set $J := J'$, $[n, v, o] := [n', v', o']$ and $k_p := 0$
9: **else**
10: Update $k_p := k_p + 1$ if needed for step 4.
11: Update $k_t := k_t + 1$ if needed for step 4.

Even though the proposal addresses a non-supervised classification technique, there is a training phase, so-called as it resembles to the training phase involved in supervised classification techniques. This phase here exploits information from the whole image and yields values for the split-operator parameters, n, v, o. This training phase can be done separately for each split operator or in an end-to-end perspective.

Some new notations are introduced.

- $[n, v, o]$ denote the best performing values of the parameters defining the S-criterion.
- $[n', v', o']$ denote exploring values of the parameters defining the S-criterion.
- J, J' are the best and the exploring values of MT.

- σ_n, σ_v are the varying standard deviation with which n' and v' are drawn, n and v being the best performing stored values.

$$E[(n'-n)^2] = \sigma_n^2 \quad E[(v'-v)^2] = \sigma_v^2 \tag{27}$$

- p_o is a varying probability with which o' is drawn, o being the also the best performing stored value.

$$P(o' = o) = p_0 \tag{28}$$

In the several classical algorithms inducing trees, features and feature-thresholds are tested in a brute-force manner. This is generally an appropriate choice since the numerical complexity of evaluating many times the cost function is not a significant issue. As for the splitting operator, I suggest using an algorithm resembling the simulated-annealing searching technique to exploit the probable similarities among nearby bandwidths and among nearby thresholds for a given bandwidth. I suggest introducing a hyper-parameter CLK to balance between numerical complexity and the exploration completeness. A gradient-descent technique would be quicker but it would be quickly stuck in local minima, and I am expecting these to be numerous.

The basic structure of the proposed training technique is described in algorithm 2, the main repeated task uses the split-operator parameters and yields a clustering map evaluated using a metric MT. The evaluations are taken into account when drawing randomly new parameter values.

The parameter values are drawn from distributions depending on three probabilistic parameters σ_n, σ_v, p_o.

$$\begin{cases} n' := \lfloor \mathrm{sat}_1^N(n + \sigma_n \mathcal{N}) \rceil \\ v' := \mathrm{sat}_0^1(v + \sigma_v \mathcal{N}) \\ o' := o\,(2 \times \mathbf{1}(\mathcal{U} > p_o) - 1) \end{cases} \tag{29}$$

where $\mathcal{N}$ is the centered normalized Gaussian distribution, $\mathcal{U}$ is the uniform distribution on $[0,1]$, $\lfloor \ldots \rceil$ is the rounding function. sat is a saturation function defined as $\mathrm{sat}_a^b(x) = \max(a, \min(x, b))$ and ensuring that $\mathrm{sat}_a^b(x) \in [a,b]$ and that $\forall x \in [a,b], \mathrm{sat}_a^b(x) = x$.

The probabilistic parameters are first initialized.

$$[\sigma_n, \sigma_v, p_o] := [\sigma_{n_0}, \sigma_{v_0}, p_{o_0}] = \left[\frac{N}{2\sqrt{3}}, \frac{N}{2\sqrt{3}}, \frac{1}{2}\right] \tag{30}$$

It is mainly how the $(\sigma_n, \sigma_v, p_o)$ parameter are varying that defines the algorithm exploring strategy. We make use of iterations counters k_p and k_t.

- k_p counts the number of non-performing iterations (J has not been improved).
- k_t counts the total number of iterations since the algorithm has begun.
- The classical Simulated Annealing uses k_t as decreasing parameter.

$$\alpha_t = 10^{-\lfloor 6\frac{k_t}{K_t} \rfloor} \quad \sigma_n = \sigma_{n_0}\alpha_t \quad \sigma_v = \sigma_{v_0}\alpha_t \quad p_o = 10^{-\lfloor 2\frac{k_t}{K_t} \rfloor} \tag{31}$$

- The algorithm, I generally use is one needing less fine tuning at the expense of increased numerical complexity.

$$\alpha = 10^{-6\mathcal{U}} \quad \sigma_n = \sigma_{n_0}\alpha \quad \sigma_v = \sigma_{v_0}\alpha \quad p_o = \frac{1}{2}\alpha \tag{32}$$

  where $\mathcal{U}$ is a uniform law on $[0,1]$.

- The algorithm, I am suggesting uses

$$\alpha = 10^{-6\frac{k_p}{20}} \quad \sigma_n = \sigma_{n_0}\alpha \quad \sigma_v = \sigma_{v_0}\alpha \quad p_o = \frac{1}{2}\alpha \tag{33}$$

  Also step 8 is changed so that there are 20 $[n, v, o]$ -triplets stored instead of only one, that is one for each exploring scale identified by the value of k_p. And equation (29) is changed accordingly using $n_{k_p}, v_{k_p}, o_{k_p}$ instead of n, v, o.

1: Set $J_{k_p} := J'$, $\left[n_{k_p}, v_{k_p}, o_{k_p}\right] := [n', v', o']$ and $k_p := 0$

Figure 10 shows the values of σ_n, σ_v and p for a randomly simulated sequence of explorations using the three different techniques close to simulated annealing. The left is the classical simulated annealing. The middle one is the one used in algorithm 1. The right one is the one I am proposing here. This graph is obtained not with an exploring sequence but one where outcomes of the exploring technique are replaced by drawing random numbers. On the graph on the right, each time the curves are climbing back, it means that an improvement has been obtained. When the curve slowly decrease, the proposed technique is using a different P_{best}, one that depends on the actual k_p, so in the space of the P-matrices, it looks like it jumps from one provisional guess to another.

7.7 Rotating operator

The task of the rotation operators is to introduce diversity among the different clustering functions by modifying the feature space.

- A simple technique is to draw randomly a subset of features as in Rotation Forest.

$$\mathrm{R}_t(\mathbf{I}_m) = \left[\mathtt{I}_{m,n_1} \dots \mathtt{I}_{m,n_{N'}}\right] \tag{34}$$

  where $[n_1 \dots n_{N'}]$ are drawn randomly without replacement. In terms of implementation, this amounts to choosing randomly an element of $\Phi_{\binom{N}{N'}}$.

- An other technique is to draw a matrix of size $N' \times N$ by drawing independently each component and to apply this matrix to $\mathtt{I}$ with a right matrix-multiplication. Note that the resulting transformation is no longer a rotation, actually increasing the diversity.

- Other techniques could be used, involving Principal Component Analysis.

7.8 Merging operator

The objective is to transform a given clustering function whose codomain is $\mathcal{F} = \{1 \dots F\}$ into a modified clustering function, $\mathtt{F}'$ whose codomain is $\mathcal{G} = \{1 \dots G\}$ knowing $G < F$ but not the ground truth function $\mathtt{G}$. We are looking for a function $\phi \in \mathcal{G}^{\mathcal{F}}$ transforming $\mathtt{F}$ into $\mathtt{F}'$: $\mathtt{F}' = \phi \circ \mathtt{F}$ There are actually $\binom{F}{G}$ different ϕ-functions and we are looking for the one optimizing the chosen metric $\mathtt{MT}$. Denoting $\Phi_{\binom{F}{G}}$ the set of such functions, the proposed technique is to find the optimal solution to

$$\widehat{\phi} = \underset{\phi \in \Phi_{\binom{F}{G}}}{\arg\min}\, \mathtt{MT}(\phi \circ \mathtt{F}) \tag{35}$$

In terms of implementation, the brute-force technique is to test each function in $\Phi_{\binom{F}{G}}$. Listing these functions in $\Phi_{\binom{F}{G}}$ amounts to finding all non-decreasing functions abiding to the constraints [1], (a non-decreasing ϕ-function is here one that fulfills $f \leq f' \Rightarrow \phi(f) \leq \phi(f')$). This is done in Octave/Matlab with the `nchoosek(1:F,G)` instruction. See in appendix C.4, the implementation used in experiment $\mathcal{E}$. However I am expecting that this brute-force technique prevents choosing F significantly greater than G.

I am proposing three merging techniques.

- The **backward search** consists in searching iteratively which cluster should be merged.

$$\begin{cases} \phi_0 = \mathbf{1}_{\mathcal{F}} \\ \phi_{t+1} = \varphi \circ \phi_t \in \Phi_{\binom{F}{F-t-1}} \\ \quad \text{with } \varphi = \underset{\varphi \in \Phi_{\binom{F-t}{F-t-1}}}{\arg\min}\ \mathtt{MT}(\varphi \circ \phi_t \circ \mathtt{F}) \\ \quad \text{and } t \in \{0 \dots F - G + 1\} \end{cases} \tag{36}$$

[1] This is actually not correct. Considering only the non-decreasing functions stems from the idea that $\mathtt{MT}$ is invariant to any one-to-one relabeling and a classical technique to avoid making unneeded tests is to restrict to non-decreasing sequences. The correct restriction is that the sequence containing the first time each value appears should be increasing.

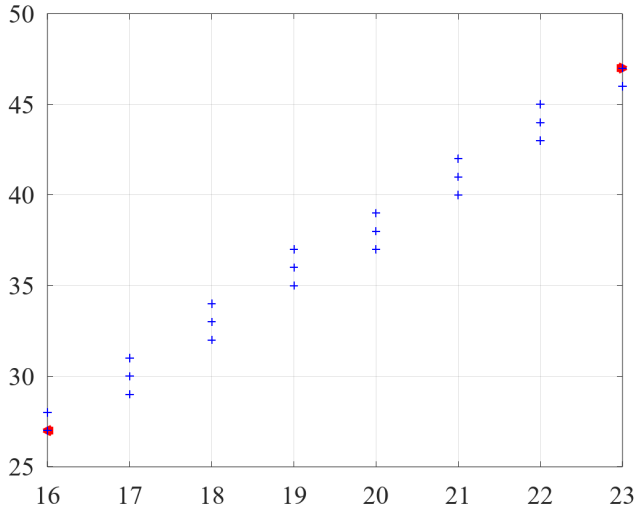


Figure 11: Illustration of the meaning of a one-pixel thick line joining m_1 and m_2 denoted as $\Gamma_{m_1}^{m_2}$ by showing with blue crosses (+) the pixels contained in $\Gamma_{m_1}^{m_2}$ where m_1 and m_2 are indicated with two red triangles.

In other words, ϕ_0 is not making any modifications to $\mathtt{F}$. ϕ_1 merges two clusters and leaves the other clusters unchanged, $\phi_1 \circ \mathtt{F}$ has $F-1$ clusters numbered from 1 to $F-1$. ϕ_2 merges two clusters each of them belonging to the clusters of $\phi_1 \circ \mathtt{F}$. $\phi_2 \circ \mathtt{F}$ has $F-2$ clusters numbered from 1 to $F-2$.

- The **neighbor grouping method** consists in searching the closest couple of neighboring clusters, in the sense that $\mathtt{MT}$ is the least augmented or most diminished when clusters f_0 and f_1 are merged. For $f_0 < f_1$, samples of cluster f_1 are put in cluster f_0.

$$(f_0, f_1) = \underset{f_0, f_1}{\arg\min} \left\{ \mathtt{MT}(\varphi \circ \mathtt{F}) \,|\varphi(f) = f\mathbf{1}(f \neq f_1) + f_0\mathbf{1}(f = f_1) \right\} \tag{37}$$

  Then we look for all G clusters partitioning F subjected to the fact that f_0 and f_1 are in the same cluster. This first technique can be further investigated by repeating the search for nearby clusters.

- The **most distant non-grouping** consists in looking for pairs of clusters (f_0, f_1) that ought not to be grouped at any cost. In other words, (f_0, f_1) is the pair of clusters that most increases $\mathtt{MT}$ when merged. Once (f_0, f_1) is selected, then we restric the search for possible clusters to those that do not put together cluster f_0 and cluster f_1. To further investigate this idea, the search for f_0, f_1 could be repeated several times to reduce the number of remaining possible grouping.

7.9 Pooling operator

It is common practice for ensemble learners to transform a set of classifiers into a single classifier that is expected to be more reliable as a mean to jeopardize *overfitting*. We are extending this idea for unsupervised classifiers by averaging the different clustering maps they yield for a specific hyperspectral image. This pooling tool is denoted as

$$\overline{\mathtt{F}} = \mathrm{POOL}_1 \left(\mathtt{F}_1, \mathtt{F}_2, \ldots, \mathtt{F}_T \right) \tag{38}$$

Section 7.9.1 describes my first idea: once we have a pixel-pixel distance, clusters are actually defined by a set of pixels. On second thoughts, it is not the idea I would use, but I did not find convincing arguments to rule it out. Section 7.9.2 are two more straightforward propositions.

7.9.1 POOL defined using a k-means algorithm and a pixel-pixel distance

Each clustering function $\mathtt{F}_t$ is first transformed into an edge-map $\partial\mathtt{F}_t$ for which 1 indicates that it is an edge and 0 not an edge. A pixel is considered as an edge if there exists a neighbor $m' \in \mathtt{N}(m)$ for which $\mathtt{F}_t$ is not equal to $\mathtt{F}(m)$.

$$\partial\mathtt{F}_t(m) = \mathbf{1}\left(\exists m' \in \mathtt{N}(m),\ \mathtt{F}_t(m) \neq \mathtt{F}_t(m') \right) \tag{39}$$

A pseudo-distance $\mathbf{d}_P(m_1, m_2)$ is then defined by counting the number of edges traversed by a one-pixel thick line joining the m_1-located pixel to the m_2-located pixel.

$$\mathbf{d}_P(m_1, m_2) = \sum_{m \in \Gamma_{m_1}^{m_2}} \left(\sum_{t=1}^{T} \partial \mathtt{F}_t(m) \right) \tag{40}$$

where $\Gamma_{m_1}^{m_2}$ is the set of pixels contained in the rectangle whose large axis is the straight line joining m_1 and m_2 and its small axis one-pixel wide and orthogonal to the large axis. An example of such a path is shown on figure 11 where the two red triangles indicate pixels m_1 and m_2 and the blue crosses (+) are the pixels in $\Gamma_{m_1}^{m_2}$. Denoting $(x_1, y_1), (x_2, y_2), (x, y)$ the coordinates of pixel m_1, m_2, m, $\Gamma_{m_1}^{m_2}$ can be defined as

$$\Gamma_{m_1}^{m_2} = \left\{ m = (x, y) \;\middle|\; \begin{array}{l} \frac{(x-x_1)(x_2-x_1)+(y-y_1)(y_2-y1)}{(x_2-x_1)^2+(y_2-y_1)^2} \in [0, 1] \\ \quad \text{and} \\ \frac{|(x-x_1)(y_2-y_1)-(y-y_1)(y_2-y1)|}{\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}} \leq 0.5 \end{array} \right\} \tag{41}$$

Finally the classical K-means algorithm is applied. In line with the literature, when a cluster m_f disappears, it

Algorithm 3 $\mathtt{K-MEANS\_POOL}$

Require: G the number of clusters

Ensure: $\mathtt{F}$

1: Compute $\mathbf{d}_P$ as in equation (40)
2: Draw G distinct pixels in $\mathcal{M}$ denoted $[m_1, \ldots, m_F]$
3: **while** $[m_1, \ldots, m_F]$ is being modified **do**
4: Set $\mathtt{F}(m) := \arg\min_{f \in \mathcal{F}} \mathbf{d}(m, m_f)$
5: Set $[m_1, \ldots, m_F]$ with $m_f := \arg\min_{m \in \mathcal{M}} \sum_{m' \in \mathtt{F}^{-1}(\{f\})} \mathbf{d}(m, m')$

makes sense to draw randomly a new one.

7.9.2 POOL defined using the existing clusters

In the two proposed techniques, we are looking for a clustering map $\mathtt{F}$ which is defined based on clusters in ${\mathtt{F}_t}^{-1}(\{g\})$ for $t \in \{1 \ldots T\}$ and $g \in \{1 \ldots G\}$ and which resembles most to all $\mathtt{F}_t$.

The first technique is just to pick up one of the clustering maps, precisely the one resembling most to all others in the sense of $\mathrm{OA}_{\mathtt{C}}$.

$$\mathrm{POOL}_2\left(\mathtt{F}_1 \ldots \mathtt{F}_T\right) = \mathtt{F}_{\bar{t}} \quad \text{where} \quad \bar{t} = \arg\max_t \sum_{t' \neq t} \mathrm{OA}_{\mathtt{C}}\left(\mathtt{F}_t, \mathtt{F}_{t'}\right) \tag{42}$$

Algorithm 4 POOL_3

Require: $\mathtt{F}_1 \ldots \mathtt{F}_T$ and G

Ensure: $\mathtt{F}$

1: Set $\Omega_0 := \mathcal{M}$,
2: **for** $g = 1 : G - 1$ **do**
3: Select the best cluster for $\mathtt{C}_g$ as in equation (45)
4: Set $\Omega_g := \Omega_{g-1} \backslash \mathtt{C}_g$
5: Set $\mathtt{C}_G := \mathcal{M} \backslash \Omega_{G-1}$
6: Collapse $\mathcal{C}_1 \ldots \mathcal{C}_G$ into $\mathtt{F}$ as in equation (44).

The second technique selects iteratively the most appropriate cluster, not taking into account pixels contained in the clusters that have already been selected. The following notations are being used.

- g happens to be also the iteration counter. There are exactly G iterations.

- $\mathcal{C}_g$ is the cluster selected at iteration g.
- Ω_g are the pixels that have not been selected throughout the g first iterations.
- $\mathcal{A} \backslash \mathcal{B}$ is the set of all pixels contained in $\mathcal{A}$ but not in $\mathcal{B}$

$$\mathcal{A} \backslash \mathcal{B} = \{m \in \mathcal{A} \,|m \notin \mathcal{B}\}$$

- $\mathcal{A} \underset{\mathcal{C}}{\triangle} \mathcal{B}$ is the ratio of size of the intersection of $\mathcal{A}$ and $\mathcal{B}$ to the size of their union, taking into account only elements in $\mathcal{C}$.

$$\mathcal{A} \underset{\mathcal{C}}{\triangle} \mathcal{B} = \frac{|\mathcal{A} \bigcap \mathcal{B} \bigcap \mathcal{C}|}{|(\mathcal{A} \bigcup \mathcal{B}) \bigcap \mathcal{C}|} \tag{43}$$

- When $\mathcal{C}_1 \dots \mathcal{C}_G$ are G disjoints clusters, they yield a clustering function:

$$\mathtt{F} = \sum_{g=1}^{G} g \mathbf{1}\left(m \in \mathtt{C}_g\right) \tag{44}$$

At each iteration, the best cluster is selected when the ratio of its intersection to its union with one existing cluster is maximized in terms of sizes. Ω_{g-1} is used here to make sure that the yielded clusters are disjoints (i.e. no pixels previously selected is included in a selected cluster).

$$\mathcal{C}_g = {\mathtt{F}_{\overline{t}}}^{-1}\left(\left\{\overline{f}\right\}\right) \bigcap \Omega_{g-1} \quad \text{where} \quad (\overline{t}, \overline{f}) = \underset{(t,f)}{\arg\max} \left(\max_{(t',f')} \left[{\mathtt{F}_t}^{-1}(\{f\}) \underset{\Omega_{g-1}}{\triangle} {\mathtt{F}_{t'}}^{-1}(\{f'\}) \right] \right) \tag{45}$$

Note that for the last cluster, there is no other choice than to select the remaining pixels, $\mathcal{C}_G := \mathcal{M} \backslash \Omega_{G-1}$.

This second technique is described in algorithm 4.

8 Proposed metrics modeling spatial continuity

A metric modeling spatial continuity maps cluster functions $\mathtt{F}$ into real values. Lower values indicate greater spatial continuity (i.e. metrics have their sign reversed when they have the opposite behavior, that is, high values indicating greater spatial continuity. Metrics are here denoted as $\mathtt{MT}$. The proposed metrics are by design invariant by any permutation of labels (they are not invariant to the fusion of clusters and hence not invariant to some Φ-mapping). I propose three kinds of metrics, the first kind considers the cluster-membership of each pixels and that of their neighbors. The second kind takes into account the connected sub-regions contained in each cluster. The third kind considers the shape of each cluster. Note that the proposed metrics are not designed to properly balance the spatial continuity within each cluster and the number of clusters. At some point, it certainly is a good idea to combine several metrics, for instance to combine the good guidance of a certain metric, with the well defined goal of an other metric. However, combining metrics may produce unexpected results, a metric may shadow the effects of the other metric. Besides, increasing the choice leads to making a lot of experiments, which takes a lot of time, it is also a cognitive bias giving the impression of having designed a performing technique, which may only work for the experiment for which it has been specifically tested, as for overfitting.

8.1 Local metrics

8.1.1 CIP: counting isolated points

CIP counts the number of isolated points.

$$\mathrm{CIP}(\mathtt{F}) = \sum_{m=1}^{M} \mathbf{1}\left(\forall m' \in \mathtt{N}(m),\ \mathtt{F}_m \neq \mathtt{F}_{m'}\right) \tag{46}$$

where $\mathtt{N}(m)$ is the set of the eight points surrounding pixel m, and $\forall$ means that all pixels m' in $\mathtt{N}(m)$ are to fulfill the property.

8.1.2 CE: counting edges

CE is the metric used in experiments $\mathcal{C}, \mathcal{D}, \mathcal{E}$ in section 6. It counts the number of edges, that is the number of points for which one of the neighbors is in a different cluster.

$$\mathrm{CE}(\mathtt{F}) = \sum_{m=1}^{M} \mathbf{1}\left(\exists m' \in \mathtt{N}(m),\ \mathtt{F}_m \neq \mathtt{F}_{m'}\right) \tag{47}$$

where $\exists$ means there exists m' in $\mathtt{N}(m)$ fulfilling that property.

The objective could be to minimize $\mathrm{CE}(\mathtt{F})$. Because we do not want to get from $\mathtt{F}$ a uniform, it might be useful to request a certain number of edge points roughly G times the number of pixels to cross the image. The new metric to be minimized might then look like

$$\mathrm{CE}_2(\mathtt{F}) = \left(\mathrm{CE}(\mathtt{F}) - G\sqrt{2|\mathcal{M}|}\right)^2$$

In light of experiment $\mathcal{B}$, we do not want this metric to guide the experiment towards a clustering similar to right of figure 4.

8.2 Metrics using connectedness

A connected region is a region for which any two points can be linked with a path contained in that region and for which any two successive points of that path are actually nearby pixels in the original image. Scientific computer languages generally include tools finding all the connected regions. If this is not available, the appendix contains the sketch of an algorithm, see algorithm 5. To explain how we are using these tools yielding clustering functions with connected clusters, we use the following notations.

- $\mathtt{F}_{\mathcal{C}} = \mathrm{FCC}(\mathtt{F})$ is the transformed clustering function.
- $\#\mathtt{F}_{\mathcal{C}}$ is the higher number of clusters of $\mathtt{F}_{\mathcal{C}}$.

$$\#\mathtt{F}_{\mathcal{C}} \geq \#\mathtt{F} = C \tag{48}$$

I am conjecturing that FCC is preserving the edges. This could be used to monitor FCC.

$$\partial(\mathtt{F}) = \partial(\mathrm{FCC}(\mathtt{F})) \tag{49}$$

I am also conjecturing that FCC could also be obtained with Edge2Map described in algorithm 6.

$$\mathrm{FCC}(\mathtt{F}) = \mathrm{Edge2Map}(\partial\mathtt{F}) \tag{50}$$

8.2.1 CNCR: counting the number of connected regions

By simply counting the number of connected regions, we may expect the algorithm to focus on the right goal.

$$\mathrm{CNCR}(\mathtt{F}) = \#\mathrm{FCC}(\mathtt{F}) \tag{51}$$

8.2.2 CPISR: counting pixels in small connected regions

CPISR counts the number of pixels contained in connected regions containing less than r pixels, r being a hyperparameter. Compared to CNCR, this metric may offer a smoother guidance.

$$\mathrm{CPISR}(\mathtt{F}) = \sum_{f=1}^{\#\mathtt{F}_{\mathcal{C}}} \mathbf{1}\left(\left|{\mathtt{F}_{\mathcal{C}}}^{-1}(\{f\})\right| \leq r\right)\left|{\mathtt{F}_{\mathcal{C}}}^{-1}(\{f\})\right| \quad \text{where} \quad \mathtt{F}_{\mathcal{C}} = \mathrm{FCC}(\mathtt{F}) \tag{52}$$

8.3 Geometrical metrics

Geometrical metrics may define more precisely what one expects of a cluster, however in terms of guidance, it could prove problematic

8.3.1 AVG_RND: measuring the average roundness of clusters

AVG_RND is intended to measure the average roundness of clusters.

The length of the f-cluster perimeter is defined as

$$P_f(\mathtt{F}) = \left|\mathtt{F}^{-1}(\{f\}) \bigcap \partial\mathtt{F}^{-1}(\{\text{EDGE}\})\right| = \sum_{m\in\mathcal{M}(\mathtt{F})} \mathbf{1}\left(\mathtt{F}(m) = f\right) \mathbf{1}\left(\partial\mathtt{F}(m) = \text{EDGE}\right) \tag{53}$$

The area of the f-cluster is the number of pixels in cluster f, it is denoted $A_f(\mathtt{F})$.

$$A_f(\mathtt{F}) = \left|\mathtt{F}^{-1}(\{f\})\right| \tag{54}$$

I propose to define the roundness of a cluster as $4\pi\frac{A_f}{P_f^2}$. If pixels were infinitely, this quantity would be small than one and it would be equal to one when the cluster is a disk. The proposed metric is an average of these values over all clusters.

$$\text{AVG\_RDN}(\mathtt{F}) = \frac{1}{\#\mathtt{F}} \sum_{f=1}^{\#\mathtt{F}} 4\pi \frac{A_f(\mathtt{F})}{P_f^2(\mathtt{F})} \tag{55}$$

8.3.2 AVG_SC: measuring the average spread of clusters

I suggest to define the spread of a cluster as the maximal distance between two points belonging to that cluster.

$$S_f(\mathtt{F}) = \max_{m,m'\in\mathtt{F}^{-1}(\{f\})} \mathbf{d}(m,m') \tag{56}$$

where $\mathbf{d}$ is the Euclidean distance between pixels.

Similarly to AVG_RND, the proposed metric is defined as the average of a ratio combining the area and the spread.

$$\text{AVG\_SC}(\mathtt{F}) = \frac{1}{\#\mathtt{F}} \sum_{f=1}^{\#\mathtt{F}} \frac{4}{\pi} \frac{A_f(\mathtt{F})}{S_f^2(\mathtt{F})} \tag{57}$$

8.3.3 MED_LSL: measuring the median size of the longest segment line contained in each cluster

I suggest to define the longest segment line contained in a cluster.

$$D'_f(\mathtt{F}) = \max_{\{m,m'|\Gamma_m^{m'}\subset\mathtt{F}^{-1}(\{f\})\}} \mathbf{d}(m,m') \tag{58}$$

where $\Gamma_m^{m'}$ is the set of all pixels close to the line segment joining m to m', it is defined in equation (41).

The proposed metric is defined as the median of D'_f along the different clusters.

$$\text{MED\_LSL}(\mathtt{F}) = \text{median}\left(D'_1(\mathtt{F}), \dots D'_{\#\mathtt{F}}(\mathtt{F})\right) \tag{59}$$

A Algorithm concerned with connectedness

A.1 FCC transforming clustering function into one whose clusters are connected

Here is a two-state pseudo-code looking for connected clusters. The first state selects a new connected region, the second state moves to a nearby location inside this connected region.

Algorithm 5 Find connected clusters (FCC)

Require: $\mathtt{F}$
Ensure: $\mathtt{F}_{\mathcal{C}}$ and $\#\mathtt{F}$
1: $\mathtt{M} := \mathcal{M}(\mathtt{F})$, $C := 1$, state := NEW_CLUSTER
2: **while** $\mathtt{M} \neq \emptyset$ **do**
3: **switch** state **do**
4: **case** NEW_CLUSTER
5: Select $m \in \mathtt{M}$ and remove it from $\mathtt{M}$
6: $\mathtt{L} := \mathtt{N}(m)$, $\mathtt{F}_{\mathcal{C}}(m) := C$, state := LOOK_AROUND
7: **case** LOOK_AROUND
8: **if** $\mathtt{L} = \emptyset$ **then**
9: state = NEW_CLUSTER
10: $C := C + 1$
11: Select $m' \in \mathtt{L}$ and remove it from $\mathtt{L}$
12: **if** $\mathtt{F}(m) = \mathtt{F}(m')$ **then**
13: $\mathtt{L} = \mathtt{L} \bigcup \mathtt{N}(m')$ and $\mathtt{F}_{\mathcal{C}}(m') := C$

Algorithm 6 Edge2map transforms an edge map into a clustering map with as many regions as needed

Require: $\partial\mathtt{F}$
Ensure: $\mathtt{F}$ and C
1: $\mathtt{M} := \mathcal{M}(\mathtt{F})$, state := NEW_CLUSTER, $C := 0$.
2: **while** $\mathtt{M} \neq \emptyset$ **do**
3: **switch** state **do**
4: **case** NEW_CLUSTER
5: **if** $\partial\mathtt{F}(m) =$ NO_EDGE **then** $C := C + 1$, state := LOOK_AROUND, $\mathtt{L} := \emptyset$
6: $\mathtt{F}(m) = C$, $\mathtt{M} := \mathtt{M} \backslash \{m\}$
7: **case** LOOK_AROUND
8: **if** $\partial\mathtt{F}(m) =$ NO_EDGE **then** $\mathtt{L} := \mathtt{L} \bigcup \mathtt{N}(m)$
9: $\mathtt{F}(m) = C$, $\mathtt{M} := \mathtt{M} \backslash \{m\}$
10: **if** $\mathtt{L} = \emptyset$ **then** state := 1
11: Select m a point in $\mathtt{L}$ and remove it form $\mathtt{L}$.

A.2 Increase_cluster: transforming edge map into clustering map

Algorithm 7 selects first the C most populous classes, the set of these classes is $\mathcal{C}$.

$$\mathcal{C} := \emptyset \quad \texttt{REPEAT} \quad \mathcal{C} := \mathcal{C} \bigcup \left\{ \arg\max_{f \notin \mathcal{C}} \left| \mathtt{F}^{-1}(\{f\}) \right| \right\} \quad \texttt{UNTIL} \quad |\mathcal{C}| = C \tag{60}$$

Denoting $\mathcal{C} = \{f_1 \dots f_C\}$, it relabels these classes as $1 \dots C$ and it assigns the remaining pixels to class 0.

$$\begin{cases} \mathtt{F}_o(m) = 1 & \text{if} \quad \mathtt{F}(m) = f_1 \\ \mathtt{F}_o(m) = 2 & \text{if} \quad \mathtt{F}(m) = f_2 \\ & \vdots \\ \mathtt{F}_o(m) = C & \text{if} \quad \mathtt{F}(m) = f_C \\ \mathtt{F}_o(m) = 0 & \text{if} \quad \mathtt{F}(m) \notin \mathcal{C} \end{cases} \tag{61}$$

For each class, it builds a binary image separating pixels belonging to that class from the other and modified by the application of the morphologic OPEN operator. This modified binary image is used to add pixels to that class, but only pixels that previously belonged to class 0. This task is repeated with square structuring elements SE of increasing sizes SIZE, until there are no more pixels in class 0.

Algorithm 7 Increase_clusters transforms a clustering map into one containing C clusters

Require: $\mathtt{F}$ and C
Ensure: $\mathtt{F}'$

1: Relabel $\mathtt{F}$ into $\mathtt{F}_o$ with equations (60) and (61).
2: Set the initial size of the structuring element SIZE := 1
3: **while** ${\mathtt{F}_o}^{-1}(\{0\}) \neq \emptyset$ **do**
4: **for** $f = 1 : C$ **do**
5: Create binary image $\mathrm{BW}_f(m) = \mathbf{1}\left(\mathtt{F}_o(m) = f\right)$ for $m \in \mathcal{M}$
6: $\mathtt{F}_o := \mathrm{OPEN}(\mathtt{F}_o, \mathrm{SE}(\mathrm{SIZE}))$
7: SIZE := SIZE + 1
8: Collapse $\mathrm{BW}_1, \dots, \mathrm{BW}_F$ into $\mathtt{F}'$ as in equation (44).

B Lemmas and their proofs

Lemma 1. *When considering any mapping of $\mathcal{F} = \{1 \dots F\}$ into $\mathcal{G} = \{1 \dots G\}$, (that is any labeling of $\mathcal{F}$-clusters in $\mathcal{G}$-classes), the maximum value of the overall accuracy that can be reached is:*

$$\max_{\phi \in \mathcal{G}^{\mathcal{F}}} OA(\mathtt{G}, \phi \circ \mathtt{F}) = \frac{1}{|\mathcal{M}|} \sum_{f=1}^{F} \max_{g} \mathtt{C}_{gf}$$

where $\phi \circ \mathtt{F}$ is the function obtained by composing ϕ with $\mathtt{F}$: $[\phi \circ \mathtt{F}](m) = \phi(\mathtt{F}(m))$

The OA-maximum value is reached when

$$\phi(f) = \arg\max_{g \leq G} \mathtt{C}_{gf}$$

Proof. Let $\phi \in \mathcal{G}^{\mathcal{F}}$. ϕ is not necessarily injective.

$$\begin{aligned} \mathrm{OA}(\mathtt{G}, \phi \circ \mathtt{F}) &= \tfrac{1}{|\mathcal{M}|} \sum_{g=1}^{G} \left| \mathtt{G}^{-1}(\{g\}) \bigcap (\phi \circ \mathtt{F})^{-1}(\{g\}) \right| = \tfrac{1}{|\mathcal{M}|} \sum_{g=1}^{G} \left| \mathtt{G}^{-1}(\{g\}) \bigcap \mathtt{F}^{-1}(\{\phi^{-1}(\{g\})\}) \right| \\ &= \tfrac{1}{|\mathcal{M}|} \sum_{g=1}^{G} \sum_{f=1}^{F} \left| \mathtt{G}^{-1}(\{g\}) \bigcap \mathtt{F}^{-1}(\{f\}) \right| \mathbf{1}(g = \phi(f)) = \tfrac{1}{|\mathcal{M}|} \sum_{g=1}^{G} \sum_{f=1}^{F} \mathtt{C}_{gf} \mathbf{1}(g = \phi(f)) = \tfrac{1}{|\mathcal{M}|} \sum_{f=1}^{F} \mathtt{C}_{\phi(f)f} \end{aligned}$$

We then see that an upperbound of $\mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F})$ is $\frac{1}{|\mathcal{M}|} \sum_{f=1}^{F} \max_g \mathtt{C}_{gf}$ and that this upperbound is reached when $\phi(f) = \arg\max_{g \leq G} \mathtt{C}_{gf}$ □

Lemma 2. *Let* $\mathtt{F}'$ *be a clustering function mapping* $\mathcal{M}$ *in* $\mathcal{F}' = \{1 \dots F'\}$ *and* $\phi' \in \mathcal{F}'^{\mathcal{F}}$.

$$\mathtt{F} = \phi' \circ \mathtt{F}' \quad \Rightarrow \quad OA_c(\mathtt{G}, \mathtt{F}') \geq OA_c(\mathtt{G}, \mathtt{F})$$

Proof. Let ϕ be the $\mathtt{G}$-optimal mapping for $\mathtt{F}$: $\phi(f) = \arg\max_{g \leq G} \mathtt{C}_{gf}$ and $\phi' \in \mathcal{F}'^{\mathcal{F}}$ be such that $\mathtt{F} = \phi' \circ \mathtt{F}'$

$$\mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F}) = \mathrm{OA}(\mathtt{G}, \phi \circ \mathtt{F}) = \mathrm{OA}(\mathtt{G}, \phi \circ \phi' \circ \mathtt{F}') \leq \max_{\phi'' \in \mathtt{G}^{\mathtt{F}'}} \mathrm{OA}(\mathtt{G}, \phi'' \circ \mathtt{F}') = \mathrm{OA}_{\mathtt{C}}(\mathtt{G}, \mathtt{F}')$$

□

C Numerical simulations

C.1 Conjecture on $\mathrm{AA}_{\mathtt{C}}$

```
function exp1()
  for k=1:1e4
    exp1a();
  end
end

function exp1a()
  M=ceil(200*rand(1));
  G=(rand(1,M)>0.5)+1;
  F=(rand(1,M)>0.5)+1;
  C=zeros(2);
  C(1,1)=sum((G==1)&(F==1)); C(1,2)=sum((G==1)&(F==2));
  C(2,1)=sum((G==2)&(F==1)); C(2,2)=sum((G==2)&(F==2));
  Phi=[0 0];
  Phi(1)=(C(1,1)>=C(2,1))+2*(C(2,1)>C(1,1));
  Phi(2)=(C(1,2)>=C(2,2))+2*(C(2,2)>C(1,2));
  Phi_F=zeros(size(F));
  Phi_F(F==1)=Phi(1);   Phi_F(F==2)=Phi(2);
  Phi_C=zeros(2);
  Phi_C(1,1)=sum((G==1)&(Phi_F==1));   Phi_C(1,2)=sum((G==1)&(Phi_F==2));
  Phi_C(2,1)=sum((G==2)&(Phi_F==1));   Phi_C(2,2)=sum((G==2)&(Phi_F==2));
  assert(sum(sum(Phi_C))==M)
  OA=sum(diag(C))/M;
  assert(OA>=0); assert(OA<=1);
  OAc=sum(diag(Phi_C))/M;
  c1=max(sum(G==1),sum(G==2));
  assert(OAc-c1/M>=0)
  if (sum(G==1)>0)&&(sum(G==2)>0)
    AA=0.5*(C(1,1)/sum(G==1)+C(2,2)/sum(G==2));
    AAc=0.5*(Phi_C(1,1)/sum(G==1)+Phi_C(2,2)/sum(G==2));
  elseif sum(G==1)>0
    AA=0.5*(C(1,1)/sum(G==1));
    AAc=0.5*(Phi_C(1,1)/sum(G==1));
  else
    AA=0.5*(C(2,2)/sum(G==2));
    AAc=0.5*(Phi_C(2,2)/sum(G==2));
  end
  assert(AA>=0), assert(AA<=1);
  assert(AAc>=0); assert(AAc>=1/2);   assert(AAc<=1);
end
```

C.2 Code of the CE-metric

```
function val=CE(im)
  B=zeros(3); B(2,2)=-1; B(1,2)=1; imngh1=(filter2(B,im)~=0);
  B=zeros(3); B(2,2)=-1; B(1,3)=1; imngh2=(filter2(B,im)~=0);
  B=zeros(3); B(2,2)=-1; B(2,3)=1; imngh3=(filter2(B,im)~=0);
  B=zeros(3); B(2,2)=-1; B(3,3)=1; imngh4=(filter2(B,im)~=0);
  B=zeros(3); B(2,2)=-1; B(3,2)=1; imngh5=(filter2(B,im)~=0);
  B=zeros(3); B(2,2)=-1; B(3,1)=1; imngh6=(filter2(B,im)~=0);
  B=zeros(3); B(2,2)=-1; B(2,1)=1; imngh7=(filter2(B,im)~=0);
  B=zeros(3); B(2,2)=-1; B(1,1)=1; imngh8=(filter2(B,im)~=0);
  imngh_ge1=(imngh1+imngh2+imngh3+imngh4+imngh5+imngh6+imngh7+imngh8>=1);
  imngh_ge1=imngh_ge1(2:(end-1),2:(end-1));
  val=sum(imngh_ge1(:))/prod(size(imngh_ge1));
end
```

C.3 Octave Code to use k-means in the special context of section 6

```
for c=1:C_plus
  KD(c,:)=data(1+(c-1)*Q,:);
end
try
  [ass, CENTERS, SUMD, DIST] = kmeans (data, C_plus,'START',KD,'DISTANCE','sqeuclidean');
catch
  disp('pb'),
  ok=0; ass=NaN;CENTERS=NaN; SUMD=NaN; DIST=NaN;  return;
end
ok=(length(unique(ass(:)))==C);
```

C.4 Simplified implementation of a merging operator used in section 6 for experiment $\mathcal{E}$

```
function lbl_l=list_lbl2(C,C_plus)
  lbl=ones(1,C_plus);
  lbl_l=ones(0,C_plus);
  pos=C_plus;
  while(~all(lbl==C*ones(1,C_plus)))
    if lbl(pos)<C
      lbl(pos)=lbl(pos)+1;
    else
      pos1=find(lbl(1:pos)<C,1,'last');
      lbl(pos1)=lbl(pos1)+1;
      lbl((1+pos1):end)=1;
      pos=C_plus;
    end
    if isnew(lbl,C)
      lbl_l=[lbl_l; lbl];
    end
  end
end

function ok=isnew(lbl,C)
  vect1=unique(lbl);
  if ~(all(diff([0 vect1])==1)) ok=0; return; end
  if ~(length(vect1)>=C) ok=0; return; end
  for val_=1:length(vect1)
```

```
    vect2(val_)=find(lbl==vect1(val_),1,'first');
  end
  ok=all(diff(vect2)>=0);
end
```